

**CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU**

Bacharelado em Ciência da Computação

**GRUPO 18**

## **GROWTH INTELLIGENCE HUB (GIH)**

Plataforma de inteligência de crescimento para redes de parceiros  
em marketplaces regionais de delivery

### **SPRINT 03 — ESTRUTURA INICIAL FUNCIONANDO**

Documento acumulado: inclui as Sprints 01 e 02

#### **EQUIPE**

Ingryd Vitoria de Araújo Barbosa — 01642893

João Pedro Nunes de França — 01626444

Marcio Maycom — 01607574

Pedro Miranda — 01607408

Thiago José Falcão de Freitas — 01597267

#### **Projeto Integrador**

Fábrica de Software · Prof.<sup>a</sup> Priscilla Gonçalves

Tópicos Avançados · Prof. Antenor Parnaíba

**2026.2**

Entrega final da disciplina: 05 de dezembro de 2026

# Sumário

## PARTE I — SPRINT 01 · PLANEJAMENTO E DESCOBERTA

1. Identificação da equipe
2. Tema
3. Definição do problema
4. Objetivos do sistema
5. Público-alvo
6. Requisitos Funcionais
7. Requisitos Não Funcionais
8. Casos de Uso
9. Product Backlog
10. Cronograma inicial
11. Repositório GitHub

## PARTE II — SPRINT 02 · ARQUITETURA E MODELAGEM

1. Arquitetura do sistema
2. Diagrama de classes
3. Modelo Entidade-Relacionamento
4. Modelo relacional
5. Protótipo das telas principais
6. Banco de dados criado
7. Projeto estruturado no GitHub

## PARTE III — SPRINT 03 · ESTRUTURA INICIAL FUNCIONANDO

1. A estrutura implementada
2. Banco de dados conectado
3. Login funcional
4. Cadastro de usuários
5. Controle de perfis
6. CRUD principal
7. Execução local
8. Repositório
9. Dificuldades encontradas
10. Ajustes no planejamento
11. Próximos passos

Documentação completa e versionada em: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

## 1. Identificação da equipe

|                 |                                                    |
|-----------------|----------------------------------------------------|
| Nome do projeto | Growth Intelligence Hub (GIH)                      |
| Turma           | CC8MB                                              |
| Disciplinas     | Fábrica de Software · Tópicos Avançados            |
| Repositório     | github.com/PedroMiranda243/gih-fabrica-de-software |

### Integrantes

Equipe de 5 integrantes, dentro do limite de 3 a 5 estabelecido pela disciplina. Os papéis organizam o trabalho, mas todos os integrantes contribuem com código e aparecem no histórico de commits do repositório.

| Integrante                       | Matrícula | GitHub           | Papel                                 |
|----------------------------------|-----------|------------------|---------------------------------------|
| Ingryd Vitoria de Araújo Barbosa | 01642893  | @ingrydaraujob   | Desenvolvedora Frontend               |
| João Pedro Nunes de França       | 01626444  | @joaopfranca04   | Banco de Dados, Documentação e Testes |
| Marcio Maycom                    | 01607574  | @mihaeldatoman   | Product Owner                         |
| Pedro Miranda                    | 01607408  | @PedroMiranda243 | Dev. Backend / Núcleo Computacional   |
| Thiago José Falcão de Freitas    | 01597267  | @ThiagojFalcão   | Scrum Master                          |

### Papéis previstos e responsabilidades

| Papel                                 | Responsabilidade principal                                     |
|---------------------------------------|----------------------------------------------------------------|
| Scrum Master                          | Cerimônias, board e acompanhamento das entregas                |
| Product Owner                         | Backlog, requisitos, priorização e critérios de aceite         |
| Dev. Backend / Núcleo Computacional   | API, regras de negócio, otimizador C++/CUDA e modelo preditivo |
| Desenvolvedor Frontend                | Telas, painel, gráficos, responsividade e acessibilidade       |
| Banco de Dados, Documentação e Testes | Modelagem, migrações, suíte de testes e documentação           |

*Os papéis organizam o trabalho; todos os integrantes contribuem com código e aparecem no histórico de commits. As histórias do núcleo em OpenMP e CUDA exigem programação em par, para que ninguém seja a única pessoa a entender o motor do projeto.*

## 2. Tema

**Growth Intelligence Hub (GIH) — plataforma de inteligência de crescimento para redes de parceiros em marketplaces regionais de delivery.**

O sistema recebe os relatórios periódicos de desempenho de uma rede de comércios parceiros e devolve decisões priorizadas: quem está crescendo, quem está prestes a cair, e — dado um orçamento e uma equipe limitados — exatamente em quais parceiros investir a próxima rodada de ações comerciais.

### Domínio de aplicação

Marketplaces regionais de delivery operam por franquia ou licença local. Uma unidade típica intermedeia pedidos entre consumidores de uma cidade ou microrregião e uma rede de 50 a 500 comércios parceiros. A unidade recebe da plataforma nacional um relatório periódico com o desempenho de cada parceiro, e é responsável por desenvolver comercialmente essa base.

### Por que este tema atende aos critérios da disciplina

| Critério                                   | Como o tema atende                                                                                                                                     |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Resolver um problema real                  | Redes regionais gerenciam a base comercial sem qualquer ferramenta analítica                                                                           |
| Complexidade compatível com um TCC         | Ingestão, modelagem temporal, regras de segmentação, aprendizado de máquina, otimização combinatória, paralelismo, GPU, controle de acesso e auditoria |
| Permitir evolução no semestre              | Sete sprints com entregas independentes, cumulativas e executáveis                                                                                     |
| Ser viável no prazo                        | Escopo fatiado por módulo; o núcleo pesado tem investigação técnica antecipada para a Sprint 3                                                         |
| Funcionalidades suficientes                | 43 requisitos funcionais em 6 módulos integrados                                                                                                       |
| Componente de IA/otimização não decorativo | O motor de decisão é o produto — sem ele o sistema vira um painel passivo                                                                              |

## 3. Definição do problema

### 3.1 A situação

Numa operação regional de delivery, a atenção comercial se concentra nos parceiros de maior faturamento — o Top 15. É uma escolha racional a curto prazo: são eles que sustentam a receita. Mas produz três efeitos que se realimentam:

- Os parceiros fora do topo — a cauda longa, tipicamente 85% da base — passam meses sem contato.
- Sem estímulo, a cauda longa não cresce, e o ranking permanece imóvel: os mesmos nomes no topo, período após período.
- Como o topo é sempre o mesmo, ele também não é desafiado — e estagna.

Ao mesmo tempo, os dados chegam em formato que não sustenta análise. O painel da plataforma mostra o número do período atual em texto corrido. Não há série histórica, não há comparação automática com o período anterior, não há alerta. O gestor vê o retrato, nunca o filme.

### 3.2 Os sub-problemas

| ID | Sub-problema                                        | Consequência operacional                                                                             |
|----|-----------------------------------------------------|------------------------------------------------------------------------------------------------------|
| P1 | Dados sem visualização analítica                    | Tendências passam despercebidas; a leitura depende de esforço manual repetido a cada período         |
| P2 | Queda de parceiro detectada tarde                   | Quando a perda fica evidente no faturamento, o parceiro já pode ter migrado ou reduzido a operação   |
| P3 | Cauda longa sem processo de relacionamento          | Ranking imóvel; potencial de crescimento não explorado em cerca de 85% da base                       |
| P4 | Alocação do esforço comercial decidida por intuição | Verba de cupom e agenda da equipe são distribuídas sem critério explícito nem verificação de retorno |
| P5 | Comunicação manual e sem segmentação                | Ações por categoria exigem esforço proporcional ao número de parceiros, e por isso não acontecem     |

### 3.3 O problema central

#### P4 é o núcleo técnico do projeto.

Escolher em quais parceiros aplicar quais ações, respeitando orçamento, capacidade da equipe e cotas por categoria, para maximizar o retorno esperado, é um problema de otimização combinatória com restrições.

O espaço de busca cresce exponencialmente: com  $N$  parceiros e  $A$  tipos de ação, existem  $(A+1)^N$  planos possíveis. Para 200 parceiros e 4 tipos de ação, isso já ultrapassa  $10^{139}$  combinações — inviável por força bruta e igualmente inviável de resolver "no olho".

É exatamente o tipo de problema que justifica metaheurística paralelizada e aceleração em GPU — e é a ponte entre as duas disciplinas: a Fábrica de Software entrega o produto completo em torno dele, Tópicos Avançados entrega o motor.

### 3.4 Por que este problema é relevante

**Para a operação.** A cauda longa concentra a maior parte do potencial de crescimento não explorado: são 85% dos parceiros recebendo uma fração da atenção comercial. Um parceiro que sai da base não é apenas receita perdida — é um comércio que passa a operar só no concorrente, e reconquistá-lo custa muito mais do que teria custado mantê-lo. O alerta que chega um período antes é a diferença entre uma conversa de retenção e uma perda consumada.

**Para os parceiros.** São comércios pequenos, quase sempre sem estrutura de marketing. Entrar numa campanha bem escolhida é acesso a demanda que não conseguiriam gerar sozinhos. Quando a alocação é feita por intuição, quem já é grande recebe mais, e a assimetria se aprofunda sozinha.

**Para a decisão.** Verba de incentivo e agenda da equipe são recursos escassos e disputados. Hoje a escolha é feita sem critério explícito e, principalmente, sem verificação posterior: não se sabe se a campanha da semana passada funcionou, porque nada foi medido contra uma previsão. Sem linha de base não existe aprendizado, apenas repetição.

**Para o campo técnico.** O problema não é uma desculpa para usar tecnologia; ele exige a tecnologia. Priorizar ações sob restrições de orçamento, capacidade e cotas é otimização combinatória, e estimar o retorno de cada ação é um problema de previsão. São duas classes de problema com solução conhecida e mensurável, o que permite avaliar objetivamente se a solução funcionou, comparando speedup e erro de previsão contra baselines, em vez de opinar sobre a interface.

3.5 O que já se sabe que não resolve

| Alternativa                | Por que não basta                                                                                                                                          |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Planilha                   | Não sustenta série histórica multi-período de centenas de parceiros, nem otimização com restrições                                                         |
| Assistente de IA genérico  | Responde sobre os dados colados, mas sem rastreabilidade, sem persistência, sem painel e sem garantia de consistência numérica entre duas perguntas iguais |
| BI genérico                | Mostra o passado. Não prevê, não otimiza e não fecha o ciclo até a ação                                                                                    |
| Ampliar a equipe comercial | Custo recorrente; não resolve o critério de priorização, apenas aumenta o volume de contatos feitos sem critério                                           |

## 4. Objetivos do sistema

### 4.1 Objetivo geral

Transformar os dados brutos de desempenho de uma rede de parceiros em um plano de ação comercial priorizado, previsto por modelo treinado a partir do histórico e otimizado sob as restrições reais de orçamento e capacidade de operação.

### 4.2 Objetivos específicos

| ID | Objetivo                                                | Verificação                                                                                  |
|----|---------------------------------------------------------|----------------------------------------------------------------------------------------------|
| O1 | Ingerir e normalizar relatórios periódicos por parceiro | Importação de 500 parceiros gera 500 registros consistentes, com período obrigatório         |
| O2 | Calcular métricas derivadas e séries históricas         | Ticket médio, variação e tendência disponíveis para qualquer parceiro com 2 ou mais períodos |
| O3 | Segmentar automaticamente por regra determinística      | A mesma base reprocessada produz exatamente a mesma segmentação                              |
| O4 | Prever faturamento e risco de queda com modelo próprio  | MAPE inferior ao de um baseline ingênuo em conjunto de teste separado                        |
| O5 | Otimizar a alocação de ações sob restrições             | O plano respeita 100% das restrições e supera a heurística "investir nos maiores"            |
| O6 | Acelerar o otimizador por paralelismo em CPU e GPU      | Speedup mínimo de 5x em GPU, com qualidade de solução equivalente                            |
| O7 | Gerar mensagens com aprovação humana obrigatória        | Nenhuma mensagem é aprovada sem ação explícita de usuário autorizado                         |
| O8 | Controlar acesso por perfil, com trilha de auditoria    | Toda ação sensível registra autor, data e parâmetros                                         |

### 4.3 Indicadores de sucesso

#### Do produto

| Indicador                       | Definição                                                     | Meta                       |
|---------------------------------|---------------------------------------------------------------|----------------------------|
| Mobilidade do Top N             | Parceiros que entram ou saem do Top 15 entre dois períodos    | Tornar visível e crescente |
| Cobertura de relacionamento     | Percentual da base que recebeu ao menos uma ação no período   | Cobertura planejada        |
| Antecedência do alerta de risco | Períodos de antecedência do sinal antes da queda se confirmar | Mínimo de 1 período        |

#### Técnicos

| Indicador                                | Meta                                  |
|------------------------------------------|---------------------------------------|
| Speedup do otimizador (GPU sobre serial) | Mínimo de 5x no cenário de referência |
| Tempo de resposta do otimizador em GPU   | Até 5 s no cenário de referência      |
| MAPE do modelo preditivo                 | Inferior ao baseline ingênuo          |
| Cobertura de testes no núcleo de regras  | Mínimo de 70%                         |

## 5. Público-alvo

### 5.1 Perfil da organização usuária

Unidade regional de marketplace de delivery — franquia ou licenciada local — com:

- 50 a 500 comércios parceiros ativos
- 1 a 3 pessoas na equipe comercial
- Relacionamento predominantemente remoto, por aplicativo de mensagens
- Sem ferramenta de BI, sem analista de dados e sem equipe de tecnologia própria
- Verba de incentivo limitada e disputada entre parceiros

### 5.2 Usuários do sistema

| Perfil              | Quem é                                         | O que faz no sistema                                      | Necessidade central                                               |
|---------------------|------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------|
| Gestor (primário)   | Responsável pela unidade; decide onde investir | Consulta o painel, executa o otimizador, aprova mensagens | Decidir rápido, com critério defensável, sem depender de analista |
| Analista comercial  | Executa o relacionamento no dia a dia          | Importa relatórios, analisa parceiros, redige mensagens   | Saber quem procurar hoje e com qual argumento                     |
| Administrador       | Responsável técnico da operação                | Gerencia usuários e perfis, ajusta parâmetros, audita     | Manter o sistema seguro e configurado sem mexer em código         |
| Parceiro (opcional) | Comércio cadastrado na rede                    | Consulta apenas o próprio desempenho                      | Enxergar o próprio resultado e a própria evolução                 |

### 5.3 Restrições impostas pelo público

O público-alvo condiciona decisões técnicas que aparecem nos requisitos não funcionais:

- Interface em português e autoexplicativa — não há treinamento formal nem suporte de TI interno (RNF20)
- Instalação em um comando — não há equipe para configurar ambiente (RNF07)
- Funcionar sem GPU — a máquina do usuário pode ser um notebook comum (RNF06)
- Rastreabilidade das análises — o gestor precisa justificar a decisão para a franqueadora e para os parceiros (RNF17)
- Aprovação humana antes de qualquer envio — a relação com o parceiro é ativo estratégico (RF39)

## 6. Requisitos Funcionais

43 requisitos funcionais distribuídos em 6 módulos. Prioridade em MoSCoW: M (indispensável), S (importante), C (desejável). Perfis: ADM Administrador, GES Gestor, ANL Analista, PAR Parceiro.

### Módulo 1 — Autenticação, perfis e auditoria

| ID   | Requisito                                                                                                         | Pri | Perfis |
|------|-------------------------------------------------------------------------------------------------------------------|-----|--------|
| RF01 | Autenticar o usuário por login e senha, criando uma sessão identificada.                                          | M   | todos  |
| RF02 | Permitir o encerramento da sessão, invalidando-a no servidor.                                                     | M   | todos  |
| RF03 | Permitir cadastrar, editar, desativar e listar usuários.                                                          | M   | ADM    |
| RF04 | Atribuir a cada usuário exatamente um perfil entre Administrador, Gestor, Analista e Parceiro.                    | M   | ADM    |
| RF05 | Restringir o acesso a cada funcionalidade conforme o perfil, validando a permissão no servidor a cada requisição. | M   | todos  |
| RF06 | Registrar em trilha de auditoria as ações sensíveis, com autor, data, hora e parâmetros.                          | M   | todos  |
| RF07 | Permitir que o usuário altere a própria senha, exigindo a senha atual.                                            | S   | todos  |
| RF08 | Permitir consultar a trilha de auditoria, com filtro por autor, tipo de ação e datas.                             | S   | ADM    |

### Módulo 2 — Ingestão e gestão de dados

| ID   | Requisito                                                                                              | Pri | Perfis        |
|------|--------------------------------------------------------------------------------------------------------|-----|---------------|
| RF09 | Importar um relatório de desempenho por período, aceitando texto colado e arquivo CSV.                 | M   | GES, ANL      |
| RF10 | Exigir a data inicial e final do período na importação e recusar a importação sem esse dado.           | M   | GES, ANL      |
| RF11 | Validar o conteúdo e exibir prévia com registros reconhecidos, rejeitados e o motivo, antes de gravar. | M   | GES, ANL      |
| RF12 | Impedir a importação de um período já registrado, oferecendo opção explícita de substituição.          | S   | GES, ANL      |
| RF13 | Manter o histórico das importações com autor, data, período coberto e total de registros.              | M   | GES, ANL, ADM |
| RF14 | Cadastrar, editar e desativar parceiros, com nome, categoria, status comercial e contato.              | M   | GES, ANL      |
| RF15 | Sugerir a categoria do parceiro pelo nome, sinalizando que é sugestão e exigindo confirmação.          | S   | GES, ANL      |
| RF16 | Oferecer gerador de dados sintéticos de 100 a 10.000 parceiros, para demonstração e benchmark.         | M   | ADM           |

### Módulo 3 — Inteligência de negócio e segmentação

| ID   | Requisito                                                                                                | Pri | Perfis        |
|------|----------------------------------------------------------------------------------------------------------|-----|---------------|
| RF17 | Exibir painel com indicadores consolidados do período e a variação frente ao período anterior.           | M   | GES, ANL      |
| RF18 | Exibir ranking por faturamento, com posição atual, posição anterior e variação percentual.               | M   | GES, ANL      |
| RF19 | Exibir a série histórica em gráfico, para a unidade e para o parceiro individual.                        | M   | GES, ANL, PAR |
| RF20 | Classificar cada parceiro em exatamente um segmento, por regra determinística com precedência explícita. | M   | GES, ANL      |

| ID   | Requisito                                                                                      | Pri | Perfis   |
|------|------------------------------------------------------------------------------------------------|-----|----------|
| RF21 | Permitir configurar os limiares da segmentação sem alteração de código.                        | S   | ADM      |
| RF22 | Exibir a mobilidade do ranking entre dois períodos, listando quem entrou e quem saiu do Top N. | M   | GES, ANL |
| RF23 | Permitir filtrar e ordenar a lista de parceiros por categoria, segmento e métricas.            | M   | GES, ANL |
| RF24 | Permitir buscar parceiro por nome, com correspondência parcial.                                | M   | GES, ANL |
| RF25 | Permitir exportar em CSV a visão atualmente filtrada.                                          | S   | GES, ANL |
| RF26 | Permitir que o perfil Parceiro consulte apenas o próprio desempenho, sem acesso a terceiros.   | C   | PAR      |

## Módulo 4 — Núcleo computacional: previsão e otimização

| ID   | Requisito                                                                                         | Pri | Perfis   |
|------|---------------------------------------------------------------------------------------------------|-----|----------|
| RF27 | Treinar modelo de previsão a partir do histórico, registrando data, volume de dados e métricas.   | M   | ADM, GES |
| RF28 | Exibir, por parceiro, o faturamento previsto e a probabilidade estimada de queda.                 | M   | GES, ANL |
| RF29 | Permitir configurar orçamento, número máximo de ações, catálogo com custos e cotas por categoria. | M   | GES      |
| RF30 | Executar o otimizador e retornar o plano de campanha, com uplift esperado e custo total.          | M   | GES      |
| RF31 | Garantir que todo plano respeite as restrições e sinalizar quando não existir solução viável.     | M   | GES      |
| RF32 | Permitir escolher o modo de execução entre serial, CPU paralelo e GPU, com seleção automática.    | M   | GES, ADM |
| RF33 | Exibir benchmark comparativo entre os modos: tempo, speedup e qualidade da solução.               | M   | GES, ADM |
| RF34 | Registrar o histórico das execuções do otimizador com parâmetros, modo, tempo e resultado.        | S   | GES, ADM |
| RF35 | Permitir comparar lado a lado dois planos de campanha gerados com parâmetros diferentes.          | C   | GES      |

## Módulo 5 — Central de comunicação

| ID   | Requisito                                                                                        | Pri | Perfis   |
|------|--------------------------------------------------------------------------------------------------|-----|----------|
| RF36 | Gerar mensagens personalizadas por segmento e categoria, a partir do plano ou de seleção manual. | M   | GES, ANL |
| RF37 | Manter as mensagens geradas em fila de aprovação, no estado pendente.                            | M   | GES, ANL |
| RF38 | Permitir aprovar, editar ou rejeitar cada mensagem antes de considerá-la pronta para envio.      | M   | GES      |
| RF39 | Impedir que qualquer mensagem seja aprovada sem ação explícita de um usuário com perfil Gestor.  | M   | GES      |
| RF40 | Manter o histórico das mensagens decididas, com autor, data e conteúdo final.                    | S   | GES, ANL |

## Módulo 6 — Assistente analítico

| ID   | Requisito                                                                                  | Pri | Perfis   |
|------|--------------------------------------------------------------------------------------------|-----|----------|
| RF41 | Responder a perguntas em linguagem natural sobre os dados armazenados.                     | S   | GES, ANL |
| RF42 | Citar o período e a origem dos dados em toda resposta, ou declarar insuficiência de dados. | M   | GES, ANL |

| ID   | Requisito                                                                                     | Pri | Perfis   |
|------|-----------------------------------------------------------------------------------------------|-----|----------|
| RF43 | Impedir que o assistente produza valores numéricos não calculados pelo núcleo determinístico. | M   | GES, ANL |

**Total: 43 requisitos funcionais — 31 Must, 9 Should, 3 Could.**

## 7. Requisitos Não Funcionais

29 requisitos não funcionais, cada um com métrica de aceitação verificável — nenhum enunciado subjetivo.

### Desempenho e escalabilidade

| ID    | Requisito                                                                                         | Métrica de aceitação                                  |
|-------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| RNF01 | O otimizador em GPU resolve o cenário de referência (2.000 parceiros, 5 ações) em uso interativo. | Até 5 s de ponta a ponta                              |
| RNF02 | O otimizador em GPU apresenta ganho sobre o baseline serial, com qualidade equivalente.           | Speedup mínimo de 5x, uplift dentro de 2% do baseline |
| RNF03 | O painel responde dentro do limiar de fluidez para bases realistas.                               | Até 2 s com 5.000 parceiros                           |
| RNF04 | O sistema suporta a carga máxima prevista sem degradação funcional.                               | 10.000 parceiros e 52 períodos                        |
| RNF05 | As consultas ao histórico usam índices, evitando varredura completa.                              | Plano sem varredura sequencial                        |

### Portabilidade e disponibilidade

| ID    | Requisito                                                               | Métrica de aceitação                          |
|-------|-------------------------------------------------------------------------|-----------------------------------------------|
| RNF06 | O sistema funciona sem GPU compatível, recorrendo ao modo CPU paralelo. | Fluxo completo executado sem GPU, com aviso   |
| RNF07 | O ambiente sobe com um único comando, sem etapa manual.                 | docker compose up a partir de clone limpo     |
| RNF08 | O sistema funciona nos navegadores de maior uso.                        | Chrome, Edge e Firefox, duas versões recentes |

### Segurança

| ID    | Requisito                                                                                              | Métrica de aceitação                     |
|-------|--------------------------------------------------------------------------------------------------------|------------------------------------------|
| RNF09 | Senhas armazenadas apenas como hash com derivação lenta e sal por usuário.                             | bcrypt ou Argon2; nenhuma senha em claro |
| RNF10 | Sessão em cookie HttpOnly e SameSite, com renovação do identificador no login.                         | Verificado por teste automatizado        |
| RNF11 | Bloqueio temporário após falhas sucessivas; resposta idêntica para usuário inexistente e senha errada. | Bloqueio após 5 falhas                   |
| RNF12 | Toda saída originada do usuário ou do modelo é escapada antes de chegar ao navegador.                  | Teste com carga de injeção de script     |
| RNF13 | Todo acesso ao banco usa consultas parametrizadas.                                                     | Revisão de código e teste de injeção SQL |
| RNF14 | A autorização por perfil é verificada no servidor em todos os endpoints.                               | Teste de cada endpoint com cada perfil   |
| RNF15 | O sistema limita o tamanho das entradas aceitas.                                                       | Limite para relatório e para pergunta    |

### Confiabilidade e rastreabilidade

| ID    | Requisito                                                                                      | Métrica de aceitação                     |
|-------|------------------------------------------------------------------------------------------------|------------------------------------------|
| RNF16 | Ranking, segmentação e métricas são determinísticos; o modelo de linguagem não calcula número. | Reprocessar produz resultado idêntico    |
| RNF17 | Toda resposta do assistente cita a origem ou declara insuficiência de dados.                   | Conjunto de perguntas de teste           |
| RNF18 | Erros retornam mensagem genérica; o detalhe fica no log do servidor.                           | Nenhum rastreamento de pilha na resposta |
| RNF19 | Log estruturado com identificador de correlação entre resposta e evento.                       | Identificador presente em ambos          |

## Usabilidade e acessibilidade

| ID    | Requisito                                                               | Métrica de aceitação                           |
|-------|-------------------------------------------------------------------------|------------------------------------------------|
| RNF20 | Interface integralmente em português e operável sem treinamento formal. | Usuário novo completa o fluxo sem documentação |
| RNF21 | Interface responsiva para desktop e tablet.                             | Layout funcional a partir de 768 px            |
| RNF22 | Texto atende ao contraste mínimo do nível AA.                           | Razão mínima de 4,5:1                          |
| RNF23 | Operações longas informam progresso.                                    | Indicador em importação e otimização           |

## Manutenibilidade e processo

| ID    | Requisito                                                                    | Métrica de aceitação                              |
|-------|------------------------------------------------------------------------------|---------------------------------------------------|
| RNF24 | O núcleo de regras tem cobertura de testes automatizados adequada.           | Cobertura mínima de 70%                           |
| RNF25 | Toda alteração chega ao ramo principal por Pull Request revisado.            | Proteção ativa; nenhum push direto                |
| RNF26 | A integração contínua executa análise estática e testes a cada Pull Request. | CI obrigatória e verde antes da mesclagem         |
| RNF27 | A documentação permite a um terceiro executar o sistema do zero.             | Validado por integrante que não escreveu o código |

## Privacidade

| ID    | Requisito                                                     | Métrica de aceitação                    |
|-------|---------------------------------------------------------------|-----------------------------------------|
| RNF28 | O sistema não coleta dados pessoais de consumidores finais.   | Modelo sem entidade de consumidor final |
| RNF29 | O repositório não contém dados reais; toda massa é sintética. | Verificação antes de cada entrega       |

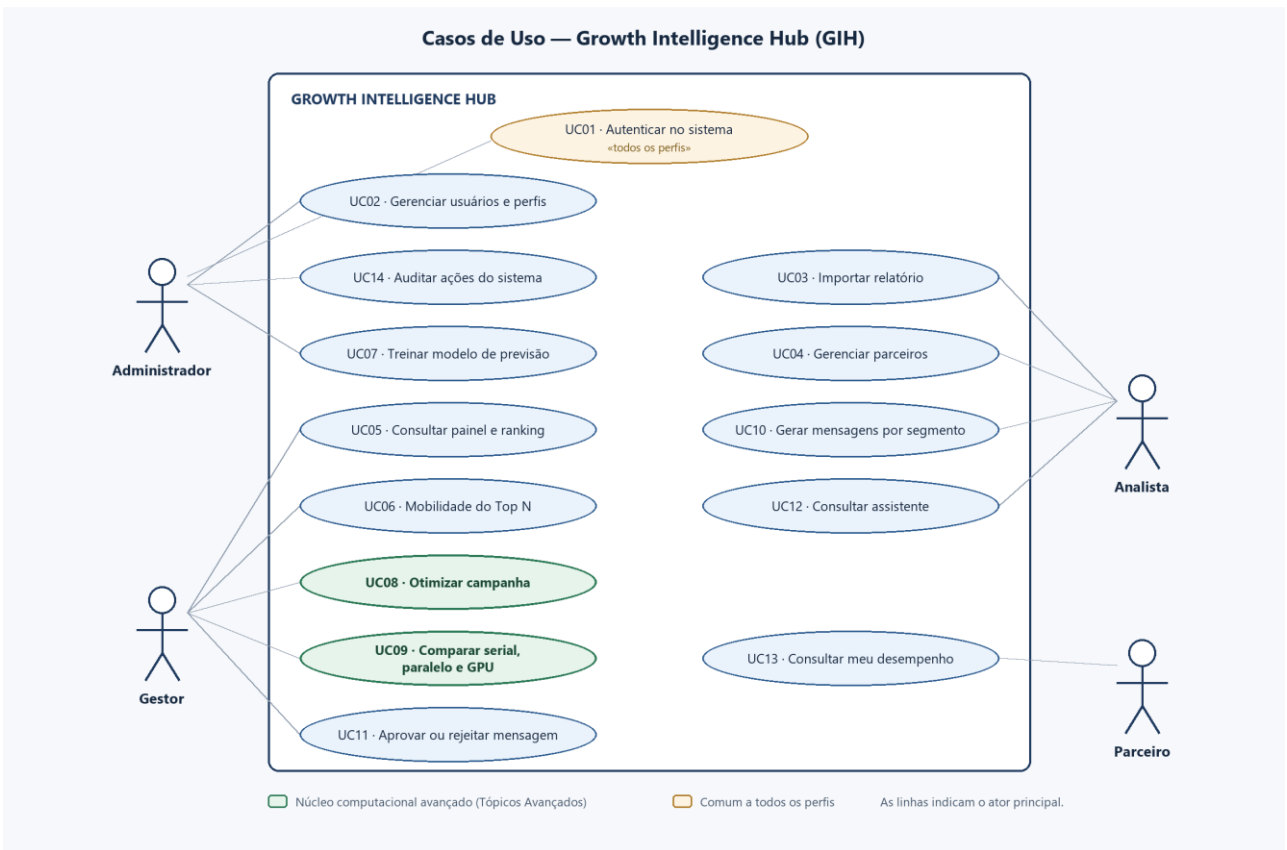
## 8. Casos de Uso

### 8.1 Atores

| Ator                      | Tipo       | Descrição                                                                                            |
|---------------------------|------------|------------------------------------------------------------------------------------------------------|
| Administrador             | Primário   | Responsável técnico da operação. Gerencia usuários, perfis e parâmetros; audita as ações registradas |
| Gestor                    | Primário   | Responsável pela unidade. Executa o otimizador, aprova mensagens e acompanha a mobilidade do ranking |
| Analista                  | Primário   | Executa o relacionamento. Importa relatórios, analisa e redige mensagens, mas não aprova envio       |
| Parceiro                  | Primário   | Comércio da rede. Acesso restrito ao próprio desempenho histórico                                    |
| Modelo de Linguagem Local | Secundário | Serviço interno que redige textos e responde ao assistente. Nunca calcula números                    |

A separação entre Gestor e Analista é o que materializa a exigência de diferentes perfis de usuários: os dois enxergam o mesmo painel, mas apenas o Gestor pode comprometer recursos (executar campanha) e comprometer a relação com o parceiro (aprovar mensagem).

### 8.2 Diagrama de casos de uso



### 8.3 Visão geral

| ID   | Caso de uso                      | Ator principal   | Requisitos cobertos |
|------|----------------------------------|------------------|---------------------|
| UC01 | Autenticar no sistema            | Todos            | RF01, RF02, RF07    |
| UC02 | Gerenciar usuários e perfis      | Administrador    | RF03, RF04, RF05    |
| UC03 | Importar relatório de desempenho | Analista, Gestor | RF09 a RF13         |
| UC04 | Gerenciar parceiros e categorias | Analista, Gestor | RF14, RF15, RF16    |

| ID   | Caso de uso                                  | Ator principal        | Requisitos cobertos      |
|------|----------------------------------------------|-----------------------|--------------------------|
| UC05 | Consultar painel e ranking                   | Gestor, Analista      | RF17 a RF20, RF23 a RF25 |
| UC06 | Analisar mobilidade do Top N                 | Gestor, Analista      | RF22                     |
| UC07 | Treinar modelo de previsão                   | Administrador, Gestor | RF27, RF28               |
| UC08 | Configurar e executar otimização de campanha | Gestor                | RF29 a RF31, RF35        |
| UC09 | Comparar desempenho serial, paralelo e GPU   | Gestor, Administrador | RF32, RF33, RF34         |
| UC10 | Gerar mensagens por segmento                 | Analista, Gestor      | RF36, RF37               |
| UC11 | Aprovar ou rejeitar mensagem                 | Gestor                | RF38, RF39, RF40         |
| UC12 | Consultar assistente analítico               | Gestor, Analista      | RF41, RF42, RF43         |
| UC13 | Consultar meu desempenho                     | Parceiro              | RF19, RF26               |
| UC14 | Auditar ações do sistema                     | Administrador         | RF06, RF08               |

## 8.4 Matriz de permissões

Legenda: X executa · L somente leitura · — sem acesso

| Caso de uso              | ADM | GES | ANL | PAR |
|--------------------------|-----|-----|-----|-----|
| UC01 Autenticar          | X   | X   | X   | X   |
| UC02 Gerenciar usuários  | X   | —   | —   | —   |
| UC03 Importar relatório  | —   | X   | X   | —   |
| UC04 Gerenciar parceiros | —   | X   | X   | —   |
| UC05 Painel e ranking    | L   | X   | X   | —   |
| UC06 Mobilidade do Top N | L   | X   | X   | —   |
| UC07 Treinar modelo      | X   | X   | —   | —   |
| UC08 Executar otimização | —   | X   | L   | —   |
| UC09 Benchmark           | X   | X   | —   | —   |
| UC10 Gerar mensagens     | —   | X   | X   | —   |
| UC11 Aprovar mensagem    | —   | X   | —   | —   |
| UC12 Assistente          | —   | X   | X   | —   |
| UC13 Meu desempenho      | —   | —   | —   | X   |
| UC14 Auditoria           | X   | —   | —   | —   |

## 8.5 Especificação detalhada — UC08: Executar otimização de campanha

| Ator principal | Gestor                                                                                            |
|----------------|---------------------------------------------------------------------------------------------------|
| Objetivo       | Obter o plano de ações que maximiza o retorno esperado dentro das restrições reais                |
| Pré-condições  | Usuário autenticado com perfil Gestor; modelo de previsão treinado; ao menos um período importado |
| Pós-condições  | Plano de campanha gerado e persistido; execução registrada no histórico e na auditoria            |
| Requisitos     | RF29, RF30, RF31, RF32, RF34, RF35                                                                |

### Fluxo principal

1. O usuário abre a tela de campanha.
2. Informa orçamento total, número máximo de ações, período de aplicação e o catálogo de ações com custo unitário.
3. Opcionalmente define cotas por categoria — por exemplo, ao menos 30% das ações destinadas à cauda longa.
4. Opcionalmente escolhe o modo de execução; se não escolher, o sistema seleciona o modo disponível mais rápido.
5. O usuário dispara a otimização.
6. O sistema recupera, para cada parceiro elegível, o faturamento previsto e o risco de queda.
7. O sistema executa o otimizador, exibindo indicador de progresso.
8. O sistema apresenta o plano de campanha: pares parceiro-ação, uplift esperado, custo total, folga por restrição e tempo de execução.
9. O usuário pode exportar o plano ou encaminhá-lo para a geração de mensagens.
10. O sistema registra a execução com autor, parâmetros, modo, tempo e resultado.

### Fluxos alternativos

- A1 — Restrições inviáveis. O sistema não retorna plano parcial: informa a inviabilidade e aponta qual restrição foi violada.
- A2 — Orçamento superior ao necessário. Retorna o plano completo, informa a sobra e sugere ampliar o catálogo de ações.
- A3 — Comparação de cenários. O usuário executa uma segunda otimização e solicita a comparação lado a lado.
- A4 — GPU indisponível. Executa em CPU paralelo e informa a substituição, sem interromper a operação.

### Exceções

- E1 — Modelo não treinado. O sistema interrompe e direciona o usuário para o caso de uso UC07.
- E2 — Execução excede o tempo limite. Preserva a melhor solução encontrada, sinaliza que é parcial e registra a ocorrência.

*Os demais casos de uso centrais — UC01, UC03, UC05, UC09 e UC11 — estão especificados no mesmo nível de detalhe no repositório, em docs/03-casos-de-uso.md.*

## 9. Product Backlog

Histórias no formato "Como <perfil>, quero <ação>, para <valor>", com prioridade MoSCoW, estimativa em story points (Fibonacci), sprint alocada e critérios de aceite. Os critérios de aceite completos estão nas issues do repositório.

### 9.1 Épicos

| Épico | Título                                | Pontos | Sprints |
|-------|---------------------------------------|--------|---------|
| E1    | Fundação, planejamento e ambiente     | 47     | 1–3     |
| E2    | Autenticação, perfis e auditoria      | 39     | 4–5     |
| E3    | Ingestão e modelo de dados            | 46     | 2–6     |
| E4    | Inteligência de negócio e segmentação | 47     | 6–9     |
| E5    | Núcleo preditivo                      | 34     | 8–9     |
| E6    | Otimização, paralelismo e GPU         | 82     | 3–11    |
| E7    | Central de comunicação                | 26     | 12      |
| E8    | Assistente analítico                  | 21     | 12–13   |
| E9    | Qualidade, documentação e entrega     | 47     | 2–13    |
|       | Total                                 | 389    |         |

### 9.2 Histórias

| ID  | História                                        | Épico | Pri | Pts | Spr |
|-----|-------------------------------------------------|-------|-----|-----|-----|
| H01 | Definir tema, problema e objetivos              | E1    | M   | 5   | 1   |
| H02 | Levantar requisitos funcionais e não funcionais | E1    | M   | 8   | 1   |
| H03 | Modelar os casos de uso                         | E1    | M   | 5   | 1   |
| H04 | Priorizar o product backlog                     | E1    | M   | 3   | 1   |
| H05 | Construir o cronograma até 05/12                | E1    | M   | 3   | 1   |
| H06 | Configurar o repositório GitHub                 | E1    | M   | 3   | 1   |
| H07 | Modelar o banco (entidade-relacionamento)       | E1    | M   | 8   | 2   |
| H08 | Prototipar as telas principais                  | E1    | M   | 5   | 3   |
| H09 | Subir o ambiente com um comando                 | E1    | M   | 5   | 2   |
| H10 | Configurar integração contínua                  | E1    | M   | 2   | 2   |
| H11 | Autenticar com login e senha                    | E2    | M   | 5   | 4   |
| H12 | Encerrar a sessão                               | E2    | M   | 2   | 4   |
| H13 | Renovar o identificador de sessão no login      | E2    | M   | 3   | 4   |
| H14 | Bloquear tentativas repetidas de login          | E2    | M   | 3   | 4   |
| H15 | Cadastrar e editar usuários                     | E2    | M   | 5   | 4   |
| H16 | Atribuir perfis de acesso                       | E2    | M   | 5   | 4   |
| H17 | Validar permissão no servidor a cada requisição | E2    | M   | 8   | 5   |
| H18 | Consultar a trilha de auditoria                 | E2    | S   | 5   | 4   |
| H19 | Alterar a própria senha                         | E2    | S   | 3   | 4   |
| H20 | Versionar o esquema do banco por migrações      | E3    | M   | 5   | 2   |
| H21 | Importar relatório colando o texto              | E3    | M   | 8   | 5   |
| H22 | Importar relatório por arquivo CSV              | E3    | M   | 3   | 6   |
| H23 | Recusar importação sem período informado        | E3    | M   | 3   | 5   |
| H24 | Ver prévia antes de gravar                      | E3    | M   | 5   | 5   |
| H25 | Alertar ao reimportar período existente         | E3    | S   | 3   | 6   |

| ID   | História                                      | Épico | Pri | Pts | Spr |
|------|-----------------------------------------------|-------|-----|-----|-----|
| H26  | Cadastrar e editar parceiros                  | E3    | M   | 5   | 3   |
| H27  | Sugerir categoria pelo nome                   | E3    | S   | 5   | 3   |
| H28  | Construir o gerador de dados sintéticos       | E3    | M   | 5   | 3   |
| H29  | Consultar o histórico de importações          | E3    | M   | 2   | 6   |
| H77  | Comando para limpar e repovoar o banco        | E3    | M   | 2   | 3   |
| H30  | Ver indicadores consolidados do período       | E4    | M   | 5   | 6   |
| H31  | Ver ranking com variação de posição           | E4    | M   | 5   | 6   |
| H32  | Ver a série histórica em gráfico              | E4    | M   | 5   | 6   |
| H33  | Segmentar por regra determinística            | E4    | M   | 8   | 7   |
| H34  | Configurar os limiares da segmentação         | E4    | S   | 3   | 7   |
| H35  | Ver quem entrou e saiu do Top N               | E4    | M   | 5   | 7   |
| H36  | Filtrar e ordenar a lista de parceiros        | E4    | M   | 5   | 7   |
| H37  | Buscar parceiro por nome                      | E4    | M   | 2   | 6   |
| H38  | Exportar a visão filtrada em CSV              | E4    | S   | 3   | 7   |
| H39  | Consultar o próprio desempenho (parceiro)     | E4    | C   | 3   | 9   |
| H40  | Painel rápido mesmo com base grande           | E4    | M   | 3   | 6   |
| H41  | Definir e extrair as variáveis preditivas     | E5    | M   | 8   | 8   |
| H42  | Treinar o modelo de previsão                  | E5    | M   | 8   | 8   |
| H43  | Estimar a probabilidade de queda              | E5    | M   | 5   | 8   |
| H44  | Ver previsão e risco na tela do parceiro      | E5    | M   | 5   | 8   |
| H45  | Disparar o retreino do modelo                 | E5    | S   | 5   | 9   |
| H46  | Comparar o modelo com baselines estatísticos  | E5    | M   | 3   | 8   |
| H47  | Validar a cadeia de compilação da GPU (spike) | E6    | M   | 5   | 3   |
| H48  | Formalizar o problema de otimização           | E6    | M   | 5   | 9   |
| H49  | Implementar o otimizador serial (baseline)    | E6    | M   | 8   | 9   |
| H50  | Configurar as restrições da campanha          | E6    | M   | 5   | 9   |
| H51  | Executar o otimizador e receber o plano       | E6    | M   | 8   | 10  |
| H52  | Recusar planos inviáveis                      | E6    | M   | 3   | 9   |
| H53a | Portar o otimizador para C++                  | E6    | M   | 8   | 10  |
| H53b | Paralelizar as partidas com OpenMP            | E6    | M   | 5   | 10  |
| H54a | Estruturas na GPU e transferência host-device | E6    | M   | 5   | 11  |
| H54b | Kernel CUDA de avaliação da população         | E6    | M   | 5   | 11  |
| H54c | Laço completo do otimizador na GPU            | E6    | M   | 3   | 11  |
| H55  | Escolher o modo de execução                   | E6    | M   | 3   | 10  |
| H56  | Cair para CPU quando não houver GPU           | E6    | M   | 3   | 10  |
| H57  | Ver o benchmark comparativo na interface      | E6    | M   | 8   | 11  |
| H58  | Consultar o histórico de execuções            | E6    | S   | 3   | 10  |
| H59  | Comparar dois planos lado a lado              | E6    | C   | 5   | 11  |
| H60  | Gerar mensagens por segmento e categoria      | E7    | M   | 8   | 12  |
| H61  | Manter fila de mensagens pendentes            | E7    | M   | 5   | 12  |
| H62  | Aprovar, editar ou rejeitar mensagem          | E7    | M   | 5   | 12  |
| H63  | Impedir aprovação sem ação humana             | E7    | M   | 5   | 12  |
| H64  | Consultar histórico de mensagens decididas    | E7    | S   | 3   | 12  |
| H65  | Perguntar sobre os dados em linguagem natural | E8    | S   | 8   | 13  |

| ID  | História                                       | Épico | Pri | Pts | Spr |
|-----|------------------------------------------------|-------|-----|-----|-----|
| H66 | Resposta citando o período usado               | E8    | M   | 5   | 13  |
| H67 | Impedir que o modelo produza números próprios  | E8    | M   | 5   | 13  |
| H68 | Assistente admite quando não sabe              | E8    | M   | 3   | 12  |
| H69 | Testes automatizados no núcleo de regras       | E9    | M   | 8   | 7   |
| H70 | Testes de segurança automatizados              | E9    | M   | 5   | 11  |
| H71 | Exigir Pull Request revisado em main           | E9    | M   | 2   | 2   |
| H78 | Teste de ponta a ponta contra a API no ar      | E9    | M   | 5   | 5   |
| H72 | Subir o sistema seguindo apenas o README       | E9    | M   | 5   | 13  |
| H73 | Documentação técnica final consolidada         | E9    | M   | 5   | 13  |
| H74 | Produzir o vídeo horizontal (até 10 min, 16:9) | E9    | M   | 8   | 13  |
| H75 | Produzir o vídeo vertical (9:16)               | E9    | M   | 5   | 13  |
| H76 | Validar acessibilidade e responsividade        | E9    | S   | 4   | 13  |

### 9.3 Primeiro corte, se o prazo apertar

Ordem de remoção acordada previamente, para que a decisão não seja tomada sob pressão:

- H59 — comparar planos lado a lado (Could)
- H39 — portal do parceiro (Could)
- H27 — sugestão de categoria (Should)
- H38 — exportação em CSV (Should)
- H65 — perguntas livres ao assistente (Should), mantendo H66, H67 e H68, que protegem a confiabilidade

**Nada dos épicos E6 e E9 entra nessa lista: são, respectivamente, o diferencial técnico avaliado e a condição de entrega.**

## 10. Cronograma

Treze sprints: a primeira já entregue e doze ciclos semanais, de segunda a sexta, até 04/12. A mudança de sprints quinzenais para semanais atende ao pedido da avaliação da Sprint 1 e traz três consequências práticas: as histórias de 13 pontos foram quebradas em etapas verificáveis, cada semana passa a ter um entregável demonstrável próprio, e um eventual atraso aparece em sete dias em vez de quatorze.

**Regra permanente: toda sprint termina com algo executável.**

| Spr | Período       | Tema                                | Entregável demonstrável                                                 |
|-----|---------------|-------------------------------------|-------------------------------------------------------------------------|
| 1   | até 05/09     | Planejamento e descoberta           | Documentação completa e repositório configurado                         |
| 2   | 14/09 – 18/09 | Modelagem e fundação                | Banco criado por migração; ambiente sobe com um comando                 |
| 3   | 21/09 – 25/09 | Dados, protótipo e spike de GPU     | Base populada por um comando; protótipo aprovado; kernel de GPU rodando |
| 4   | 28/09 – 02/10 | Autenticação e usuários             | Login com sessão; cadastro de usuários e perfis                         |
| 5   | 05/10 – 09/10 | Autorização e ingestão por texto    | Perfis barrados no servidor; relatório importado com prévia             |
| 6   | 12/10 – 16/10 | Ingestão completa e painel          | CSV, histórico e painel com indicadores e ranking                       |
| 7   | 19/10 – 23/10 | Segmentação e mobilidade do Top N   | Cada parceiro segmentado; quem entrou e saiu do Top N                   |
| 8   | 26/10 – 30/10 | Modelo preditivo                    | Modelo treinado, MAPE melhor que o baseline, previsão na tela           |
| 9   | 02/11 – 06/11 | Otimizador: formalização e baseline | Otimizador serial devolvendo plano válido                               |
| 10  | 09/11 – 13/11 | Otimizador integrado e CPU paralela | Plano na interface; versão C++ com OpenMP e ganho medido                |
| 11  | 16/11 – 20/11 | GPU e benchmark                     | Speedup medido e exibido na tela                                        |
| 12  | 23/11 – 27/11 | Central de comunicação              | Mensagens geradas e fila de aprovação funcionando                       |
| 13  | 30/11 – 04/12 | Assistente e fechamento             | Sistema completo, documentado e os dois vídeos                          |

**Carga:** 389 pontos no total, 27 já entregues na Sprint 1 e 362 distribuídos em doze semanas — média de 30,2 pontos por sprint, contra uma capacidade nominal de cerca de 40. A Sprint 13 está deliberadamente acima da média, e isso está tratado em 10.3.

### 10.1 Marcos

| Data  | Marco                            | Critério de verificação                                     |
|-------|----------------------------------|-------------------------------------------------------------|
| 05/09 | Planejamento entregue            | Documento no Teams e formulário de identificação preenchido |
| 25/09 | Risco de GPU retirado            | Kernel CUDA compilado e conferido contra a CPU              |
| 02/10 | Acesso controlado                | Login com 4 perfis, negação validada no servidor            |
| 16/10 | Dados entrando e painel no ar    | Importação completa e ranking com variação                  |
| 23/10 | Inteligência de negócio pronta   | Segmentação determinística e mobilidade do Top N            |
| 30/10 | Modelo batendo o baseline        | MAPE registrado e inferior ao baseline ingênuo              |
| 13/11 | Otimizador paralelo em CPU       | Plano válido e ganho medido sobre o serial                  |
| 20/11 | Componente avançado demonstrável | Speedup medido e exibido na interface                       |
| 27/11 | Produto completo                 | Fluxo inteiro, do dado bruto à mensagem aprovada            |
| 30/11 | Congelamento de escopo           | Nenhuma funcionalidade nova a partir desta data             |

| Data  | Marco         | Critério de verificação                               |
|-------|---------------|-------------------------------------------------------|
| 05/12 | Entrega final | Sistema, código, documentação, banco e os dois vídeos |

## 10.2 Riscos e mitigações

| #  | Risco                                                              | Prob. | Mitigação                                                                                                                                                                                          |
|----|--------------------------------------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 | A cadeia de compilação de GPU não funcionar no ambiente disponível | Média | Spike antecipado para a Sprint 3, oito semanas antes de ser necessário. Alternativas em ordem: CuPy, Numba com destino CUDA, OpenCL                                                                |
| R2 | Escopo completo sem folga no calendário                            | Alta  | A média de 30 pontos por semana não deixa margem. Válvula de escape definida em 10.3: o assistente analítico sai primeiro. A velocidade real das Sprints 2 e 3 recalibra o plano antes da Sprint 8 |
| R3 | O speedup em GPU ficar abaixo da meta de 5x                        | Média | Ampliar a escala do cenário com o gerador sintético. Se ainda assim não atingir, reportar o resultado medido com a análise do porquê                                                               |
| R4 | Trilha do núcleo concentrada em uma pessoa                         | Alta  | E5 e E6 somam 116 pontos sob o mesmo responsável, sem par. Mitigação por transferência de conhecimento: decisões em ADR e apresentação do código do núcleo nas revisões das Sprints 10 e 11        |
| R5 | Modelo preditivo não superar o baseline ingênuo                    | Média | Baselines implementados antes do modelo, para saber cedo qual é o alvo; o otimizador segue com a melhor estimativa disponível                                                                      |
| R6 | Ausência ou queda de participação de um integrante                 | Média | O ciclo semanal expõe o problema em sete dias; acompanhamento por commits e board em cada orientação                                                                                               |
| R7 | Vídeos deixados para os últimos dias                               | Alta  | A Sprint 13 já nasce sobrecarregada. Roteiros escritos na Sprint 12; congelamento em 30/11; gravação distribuída entre 01/12 e 04/12                                                               |
| R8 | Escopo crescer durante o semestre                                  | Média | Toda ideia nova entra no backlog como história e disputa prioridade; não é incorporada direto à sprint em andamento                                                                                |

## 10.3 A semana que não fecha

A Sprint 13 está com 45 pontos contra uma média de 30, e é justamente a semana dos dois vídeos. Não é erro de distribuição: é o que sobra quando o escopo completo é dividido por doze semanas, com as outras onze já entre 22 e 32 pontos.

Registrar isso agora tem um propósito: quando a pressão chegar, a decisão já estará tomada em vez de improvisada. O assistente analítico é a válvula de escape — se qualquer semana entre a 8 e a 12 atrasar, ele é o que não se constrói. A justificativa é que ele não é o componente de inteligência avaliado: esse papel cabe ao modelo preditivo e ao otimizador paralelo.

**O que não pode sair em nenhuma hipótese: os dois vídeos, a documentação final e a reprodutibilidade a partir do README. Sem eles não há entrega.**

*As doze sprints semanais foram distribuídas a partir das duas datas oficiais divulgadas: entrega da Sprint 1 em 05/09 e entrega final em 05/12. O calendário de orientações da disciplina tem precedência: o cronograma será ajustado preservando a sequência de temas e os marcos técnicos.*

## 11. Repositório GitHub

Endereço: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

### 11.1 O que já está configurado

| Recurso                  | Situação                                                                       |
|--------------------------|--------------------------------------------------------------------------------|
| Visibilidade             | Público                                                                        |
| Documentação             | Sete documentos em docs/, cobrindo todos os entregáveis desta sprint           |
| Diagrama de casos de uso | Publicado em SVG, renderizando diretamente no GitHub                           |
| Labels                   | Por épico (E1 a E9), prioridade (MoSCoW) e tipo (feat, fix, docs, test, spike) |
| Milestones               | Sete, uma por sprint, com data de encerramento                                 |
| Issues                   | Histórias das Sprints 1 a 3 abertas, com critérios de aceite                   |
| Modelos                  | Issue (história e defeito) e Pull Request, com Definition of Done              |
| Proteção do ramo main    | Exige 1 aprovação em Pull Request; bloqueia push forçado e exclusão            |
| Contribuição             | CONTRIBUTING.md com padrão de ramos, commits e Definition of Done              |

### 11.2 Estrutura do repositório

- README.md — problema, solução, stack, execução e equipe
- CONTRIBUTING.md — fluxo de branches, commits e Pull Requests
- docs/01-visao-do-produto.md — tema, problema, objetivos e público-alvo
- docs/02-requisitos.md — requisitos funcionais e não funcionais, com rastreabilidade
- docs/03-casos-de-uso.md — diagrama e especificação dos casos de uso
- docs/04-product-backlog.md — épicos e histórias priorizadas
- docs/05-cronograma.md — sprints, marcos e riscos
- docs/06-equipe-e-processo.md — papéis, cerimônias e Definition of Done
- docs/07-arquitetura-preliminar.md — visão de contêineres, stack e decisões de arquitetura

### 11.3 Regras de dados e confidencialidade

O repositório é público, e por isso valem sem exceção:

- Nenhum dado real de nenhuma operação. Toda massa de demonstração é gerada por script.
- Nenhum segredo versionado: senhas e tokens ficam em .env, bloqueado pelo .gitignore.
- Nenhum nome de organização, pessoa ou parceiro real em código, documentação ou dados de exemplo.
- Verificação de conteúdo sensível executada antes de cada entrega.

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*

## Parte II — Sprint 02: Arquitetura e Modelagem

Esta parte apresenta a estrutura técnica que sustenta o desenvolvimento do sistema: como a solução está organizada, como as classes se relacionam, como os dados são modelados, como as telas se comportam e qual é o estado atual do banco e do repositório.

Tudo aqui decorre do que foi definido na Parte I. O problema, os objetivos e os requisitos não mudaram — o que muda é que agora existe uma arquitetura que responde a eles, e código rodando que a comprova.

### Duas numerações de sprint

A equipe trabalha em sprints semanais, e a disciplina avalia por sprints de entrega. Como os dois sistemas de numeração convivem na documentação do repositório, vale deixar a correspondência explícita antes de qualquer outra coisa.

| Entrega da disciplina                 | Prazo      | Sprints da equipe | O que cobre                                                              |
|---------------------------------------|------------|-------------------|--------------------------------------------------------------------------|
| Sprint 01 — planejamento e descoberta | 05/09/2026 | 1                 | Tema, problema, objetivos, requisitos, casos de uso, backlog, cronograma |
| Sprint 02 — arquitetura e modelagem   | 19/09/2026 | 2 a 5             | Arquitetura, classes, MER, modelo relacional, protótipo, banco criado    |

**A segunda entrega cobre quatro das nossas sprints, e não uma.** Na data desta entrega o banco já está criado e migrado, a API já autentica com perfis e já ingere relatórios, e o risco técnico do núcleo em GPU já foi retirado por medição.

# 1. Arquitetura do sistema

O sistema se organiza em cinco componentes com responsabilidades separadas. A separação não é estética: ela é o que permite medir o núcleo computacional isoladamente, trocar o modelo preditivo sem tocar na interface, e garantir que o assistente de linguagem nunca produza um número.

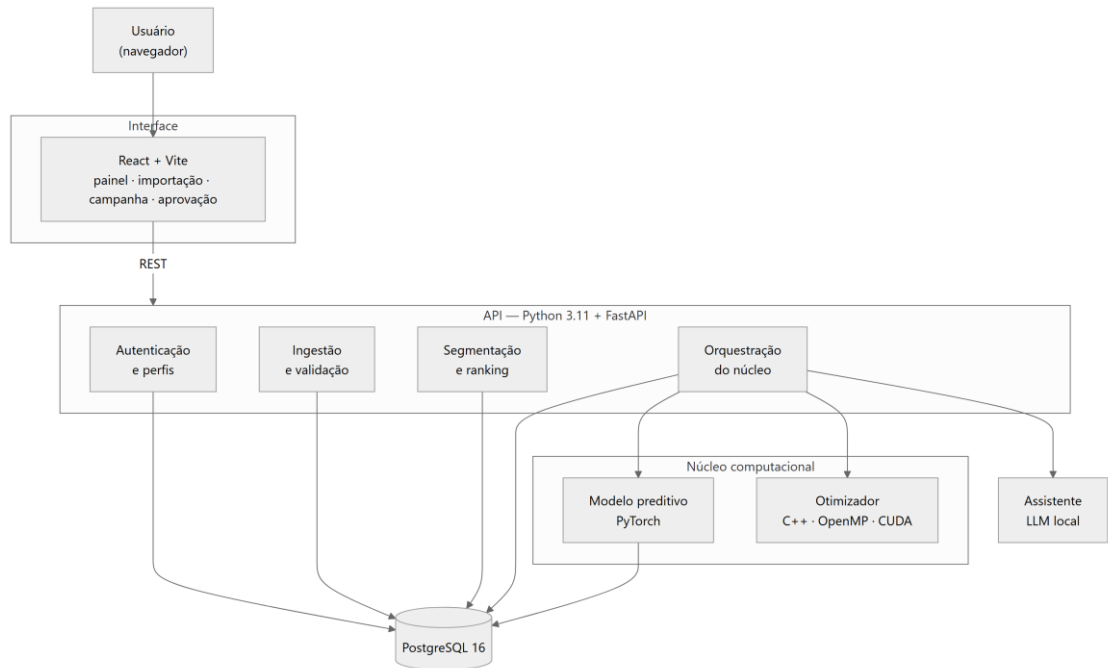


Figura 1 — Arquitetura geral: interface, API, banco, núcleo computacional, modelo preditivo e assistente.

## 1.1 Componentes e responsabilidades

| Camada     | Responsabilidade                                           | O que não faz                       |
|------------|------------------------------------------------------------|-------------------------------------|
| web/       | Exibe, formata e coleta entrada                            | Nenhuma regra de negócio            |
| api/       | Regras de negócio, autorização, persistência, orquestração | Cálculo pesado dentro da requisição |
| nucleo/    | Otimizador combinatório: serial, OpenMP e CUDA             | Não conhece autenticação nem banco  |
| modelo/    | Previsão de faturamento e probabilidade de queda           | Não decide ação comercial           |
| assistente | Redige texto sobre fatos já apurados                       | Não calcula número                  |

## 1.2 Tecnologias previstas

| Camada     | Tecnologia                           | Por quê                                                                       |
|------------|--------------------------------------|-------------------------------------------------------------------------------|
| Interface  | React + Vite                         | Componentização e recarga rápida no desenvolvimento                           |
| API        | Python 3.11 + FastAPI                | Tipagem em tempo de execução e especificação da API gerada automaticamente    |
| Banco      | PostgreSQL 16 + Alembic              | Restrições de integridade no banco e evolução do esquema versionada           |
| Núcleo     | C++17 + OpenMP + CUDA                | Paralelismo real em CPU e GPU — é o componente avaliado por Tópicos Avançados |
| Modelo     | PyTorch + NumPy                      | Treino local, sem serviço externo                                             |
| Assistente | Modelo de linguagem local via Ollama | Roda na própria máquina: sem custo por uso e sem enviar dados para fora       |

| Camada        | Tecnologia     | Por quê                                |
|---------------|----------------|----------------------------------------|
| Empacotamento | Docker Compose | O ambiente inteiro sobe com um comando |

A disciplina orienta que JavaScript não seja a tecnologia do núcleo computacional. A arquitetura respeita isso integralmente: JavaScript existe apenas na interface. Inteligência artificial, processamento intensivo, paralelismo, otimização e GPU estão em Python, C++ e CUDA.

### 1.3 Como os componentes se comunicam

- A interface conversa com a API por REST, em JSON, sobre HTTP. Nenhum outro componente é acessível a partir do navegador.
- A API é o único componente que fala com o banco. Toda leitura e escrita passa por consulta parametrizada.
- A API entrega ao núcleo um cenário já montado — uplift, custo e categoria por parceiro — e recebe de volta um plano e o tempo de execução.
- A API entrega ao modelo preditivo a série histórica e recebe previsão por parceiro.
- A API entrega ao assistente um bloco de fatos já calculados; o assistente devolve texto, nunca número.

### 1.4 A integração com o componente computacional avançado

O otimizador resolve um problema combinatório: dados N parceiros e A ações possíveis, escolher no máximo uma ação por parceiro de forma a maximizar o uplift esperado sem violar o orçamento, o número máximo de parceiros e a cota por categoria. O espaço de busca tem  $(A+1)^N$  combinações — inviável por força bruta na escala de referência de 2.000 parceiros.

A solução é uma metaheurística populacional escrita pela equipe, com três implementações da mesma interface: serial, paralela em CPU com OpenMP e paralela em GPU com CUDA. As três produzem o mesmo tipo de resultado, o que é o que torna o benchmark comparável.

**O risco técnico desse componente já foi retirado, por medição.** Uma investigação técnica dedicada validou a cadeia de compilação e mediu o ganho real oito semanas antes de ser necessário. O kernel CUDA chega a 100x sobre o baseline serial, e o resultado confere com o cálculo em CPU com erro relativo zero.

A medição também produziu um achado que mudou o projeto: comparada com a CPU paralela, e não com a serial, a GPU só compensa se a população permanecer na memória dela entre as gerações. Com transferência a cada geração o ganho cai para 1,1x; sem ela, chega a 11x. Isso deixou de ser otimização e virou requisito de projeto do componente.

### 1.5 Decisões de arquitetura registradas

Cada decisão não óbvia está registrada com a situação que a motivou, as alternativas consideradas e as consequências assumidas. São nove até aqui:

| ADR | Decisão                                                            | Consequência assumida                                                   |
|-----|--------------------------------------------------------------------|-------------------------------------------------------------------------|
| 001 | Python + C++/CUDA como stack do núcleo                             | Duas linguagens no projeto, em troca de paralelismo real                |
| 002 | Metaheurística paralela, não solver pronto                         | Mais trabalho; é o componente que a disciplina avalia                   |
| 003 | Segmentação e ranking determinísticos, fora do modelo de linguagem | Reprocessar a mesma base dá o mesmo resultado                           |
| 004 | Degradação para CPU quando não houver GPU                          | O sistema nunca deixa de funcionar por falta de hardware                |
| 005 | CUDA via NVRTC, sem instalar o toolkit                             | Superada pela 006 no mesmo dia; mantida como registro                   |
| 006 | Toolchain nativo: compilar com nvcc e MSVC                         | Quem mexe no núcleo precisa do compilador instalado                     |
| 007 | Sessão com estado no servidor, não token autocontido               | Uma consulta a mais por requisição; encerrar sessão tem efeito imediato |

| ADR | Decisão                                                        | Consequência assumida                                     |
|-----|----------------------------------------------------------------|-----------------------------------------------------------|
| 008 | A auditoria grava em transação própria                         | Registro a mais é ruído; registro a menos é ponto cego    |
| 009 | Formato do relatório importado tolerante, guiado por cabeçalho | Tolerância esconde erro; mitigada pela mensagem de recusa |

## 2. Diagrama de classes

O sistema tem três camadas com naturezas diferentes: entidades de domínio, serviços com a regra de negócio, e o núcleo computacional em C++ e CUDA. Apresentá-las num único desenho produziria uma figura de oitenta caixas, ilegível impressa. São oito diagramas, cada um legível sozinho, e o primeiro mostra como eles se ligam.

### 2.1 Visão de integração

Onde cada camada vive e o que atravessa a fronteira entre elas.

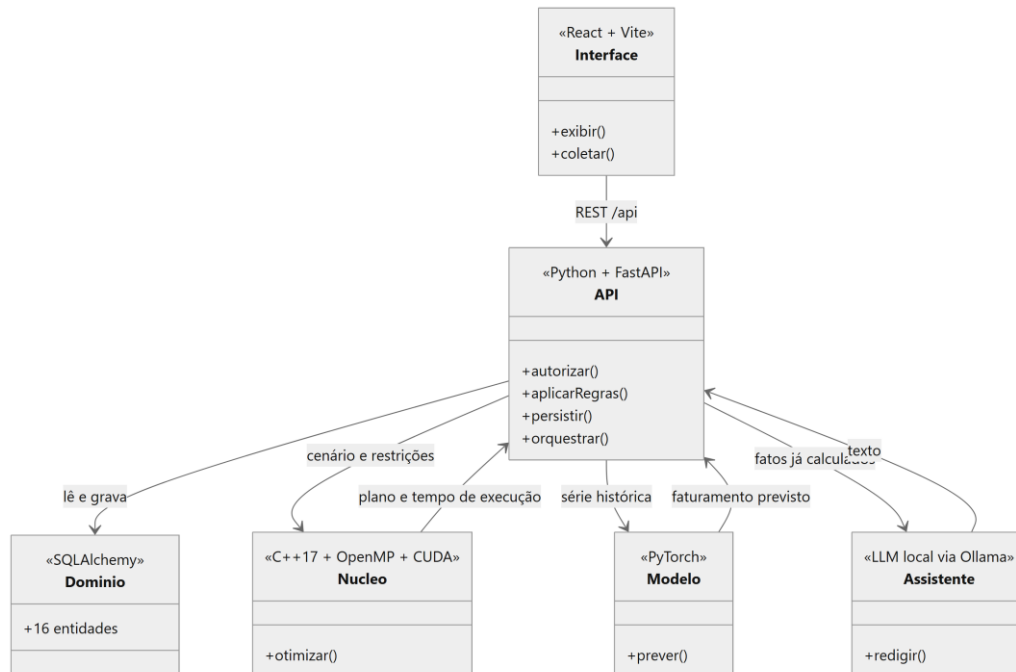


Figura 2 — Visão de integração entre as camadas, com as tecnologias de cada uma.

Três fronteiras não se atravessam: a interface não conhece regra de negócio, o núcleo não conhece autenticação nem banco, e o assistente não calcula número.

## 2.2 Domínio — acesso, sessões e auditoria

Quem entra no sistema, por quanto tempo, e o rastro que fica.

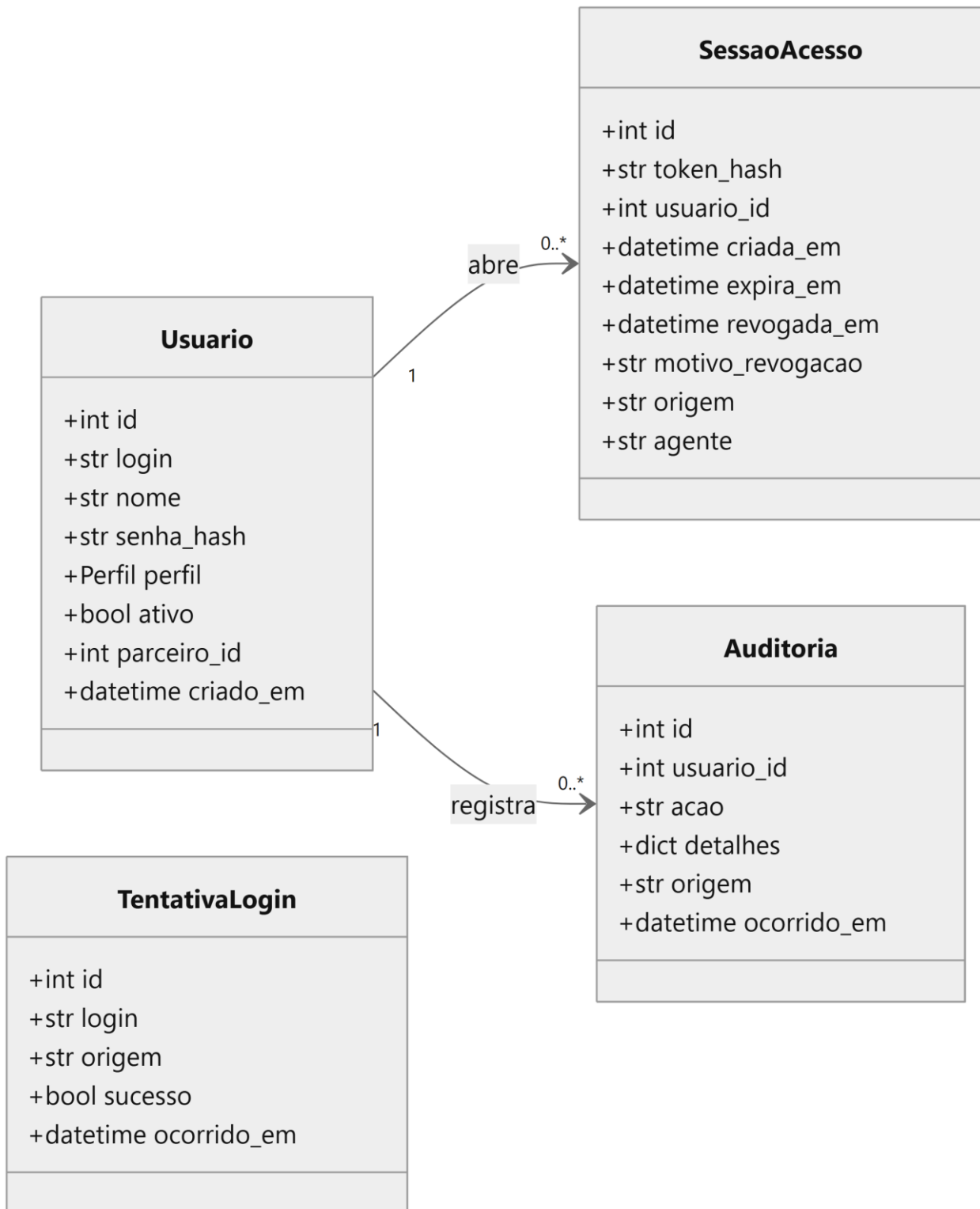


Figura 3 — Entidades de controle de acesso: usuário, sessão, tentativas de login e trilha de auditoria.

A sessão guarda o hash do identificador, nunca ele próprio: quem conseguir ler a tabela não consegue se passar por ninguém. E TentativaLogin não se liga a Usuario de propósito — o login é texto livre, porque tentativa contra usuário inexistente também precisa ser contada. É justamente o login desconhecido que o ataque por dicionário usa.

## 2.3 Domínio — parceiros e dados de desempenho

O núcleo informacional do produto: quem são os parceiros, o que faturaram, e como foram classificados.

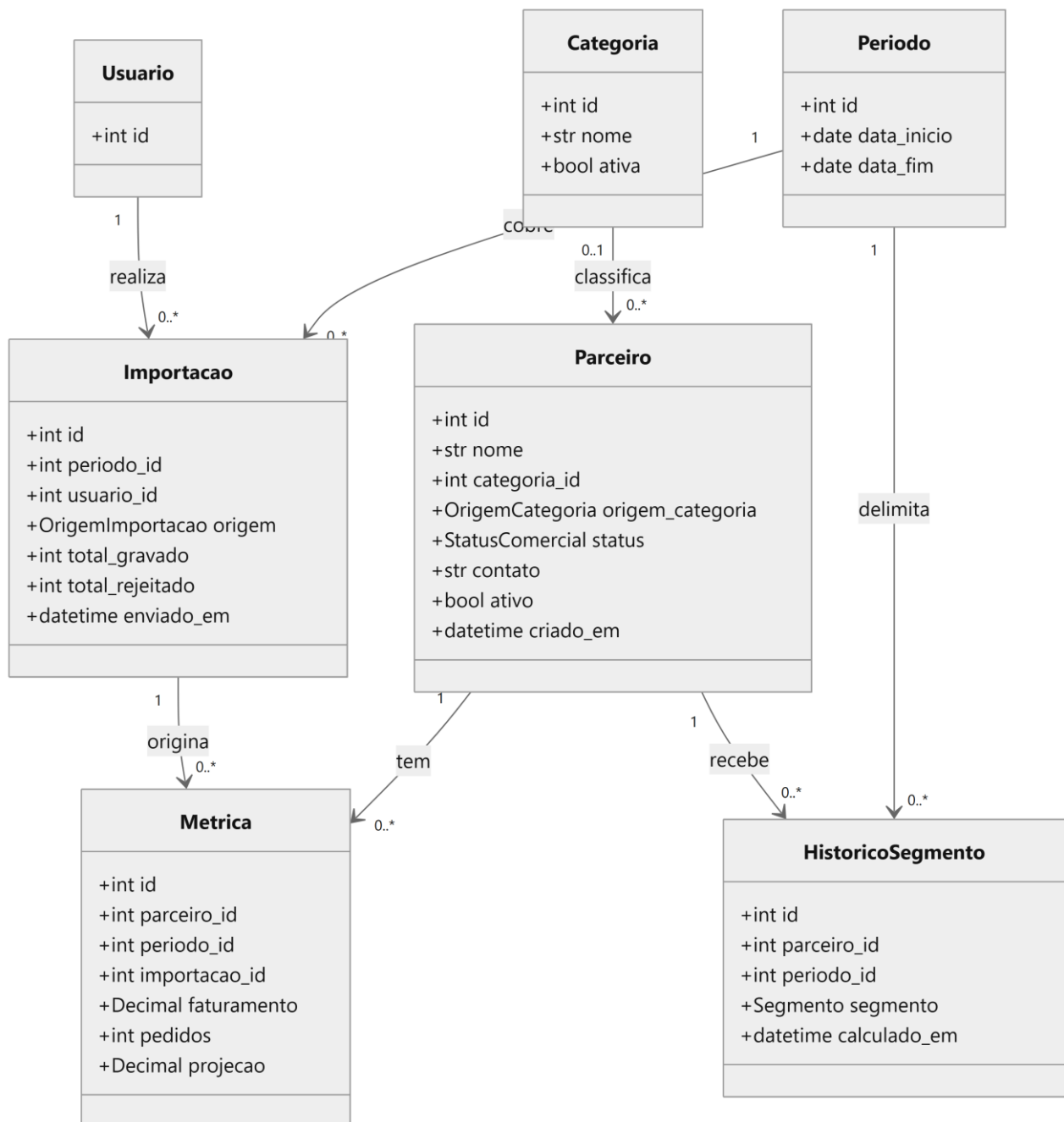


Figura 4 — Entidades de negócio: categoria, parceiro, período, importação, métrica e histórico de segmento.

Métrica não tem atributo de ticket médio: ele é faturamento dividido por pedidos, calculado na consulta. Como coluna, divergiria das parcelas que o originam na primeira correção de dado.

## 2.4 Domínio — núcleo computacional e comunicação

As entidades que registram previsões, execuções do otimizador, planos de campanha e mensagens.

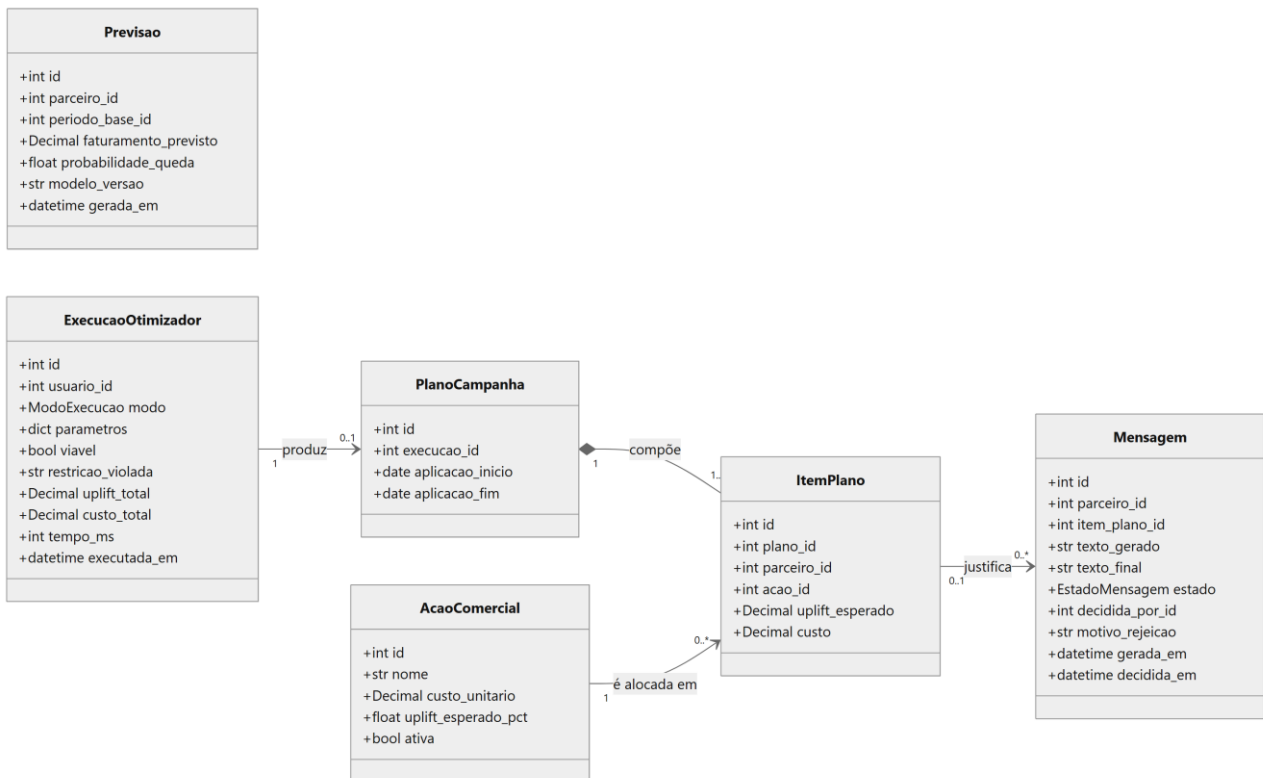


Figura 5 — Entidades do núcleo e da comunicação: previsão, ação, execução, plano, item e mensagem.

A execução do otimizador produz zero ou um plano: ou o plano respeita todas as restrições, ou não existe plano. A execução inviável fica registrada com o motivo, porque explicar por que não deu é mais útil que sumir com a tentativa.

## 2.5 Serviços — acesso e segurança

As entidades acima são anêmicas de propósito: carregam dados e restrições, e a regra de negócio vive nos serviços. A razão é testabilidade — a regra fica exercitável sem instanciar entidade.

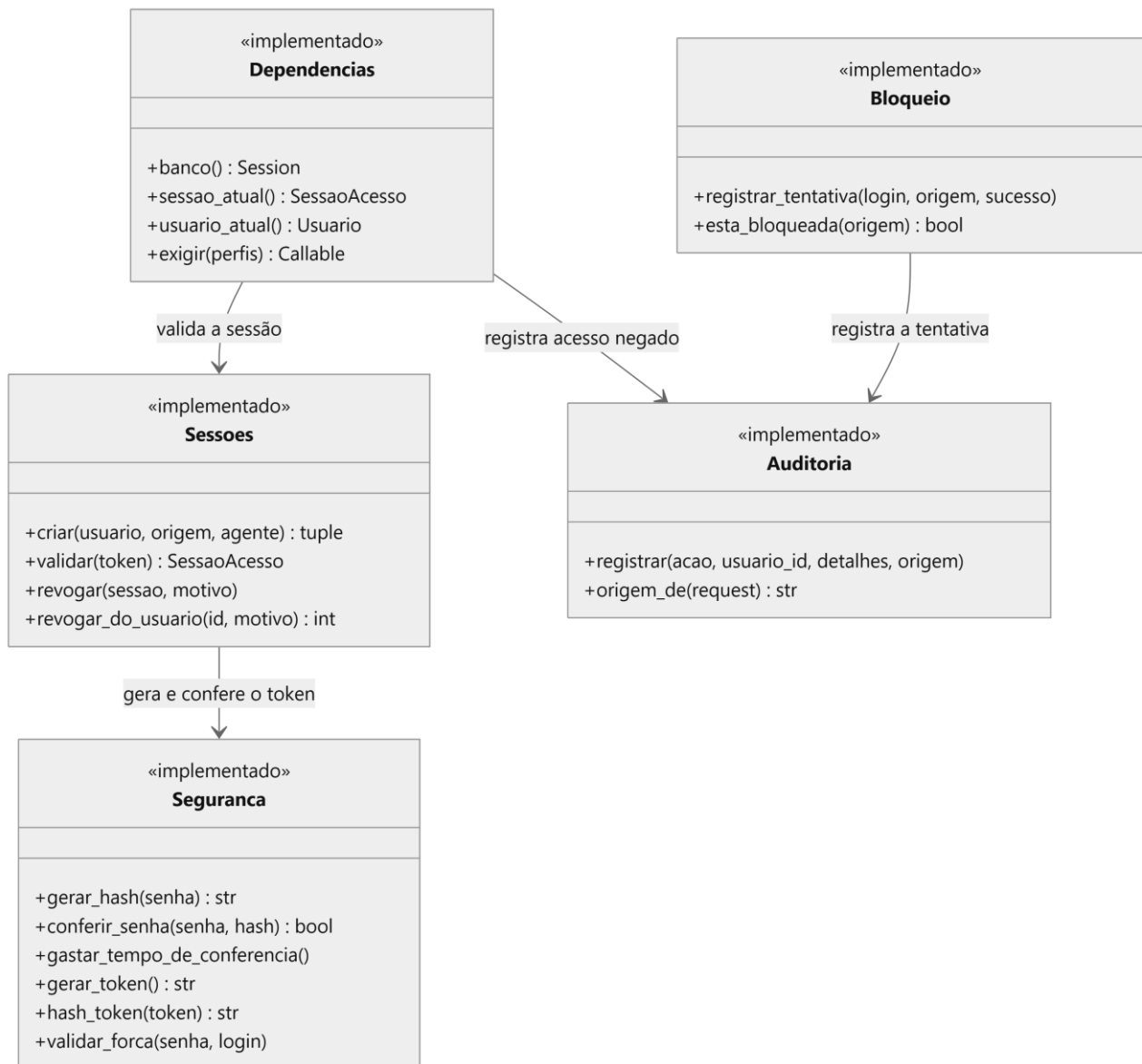


Figura 6 — Serviços de autenticação, sessão, bloqueio por força bruta e auditoria. Todos implementados.

A autorização é dependência declarada na rota, e não uma verificação dentro de cada função: esquecer uma verificação é silencioso, e esquecer a dependência reprova o teste que percorre cada endpoint contra cada perfil.

## 2.6 Serviços — ingestão e análise

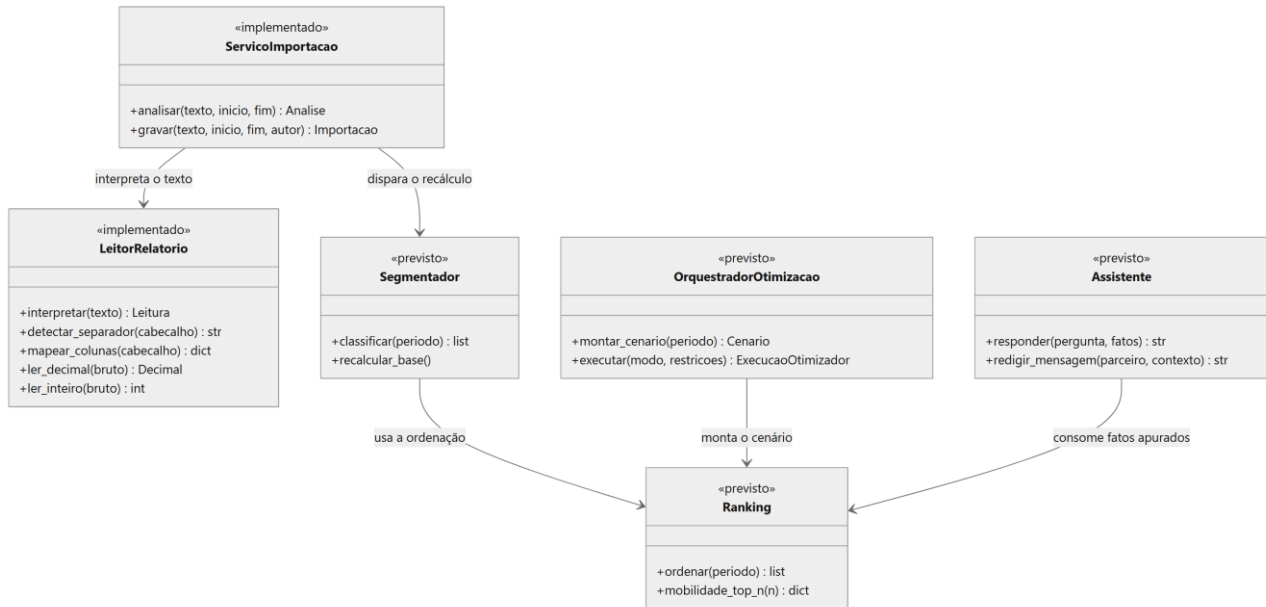


Figura 7 — Serviços de ingestão, segmentação, ranking, otimização e assistente. Os dois primeiros implementados.

O interpretador do relatório não toca no banco: é função pura de texto para resultado. É isso que permite a prévia da importação usar exatamente o mesmo código da gravação, sem risco de a prévia mostrar uma coisa e a gravação fazer outra.

## 2.7 Núcleo computacional — as estruturas

O que entra no otimizador, o que sai, e quem avalia um candidato.

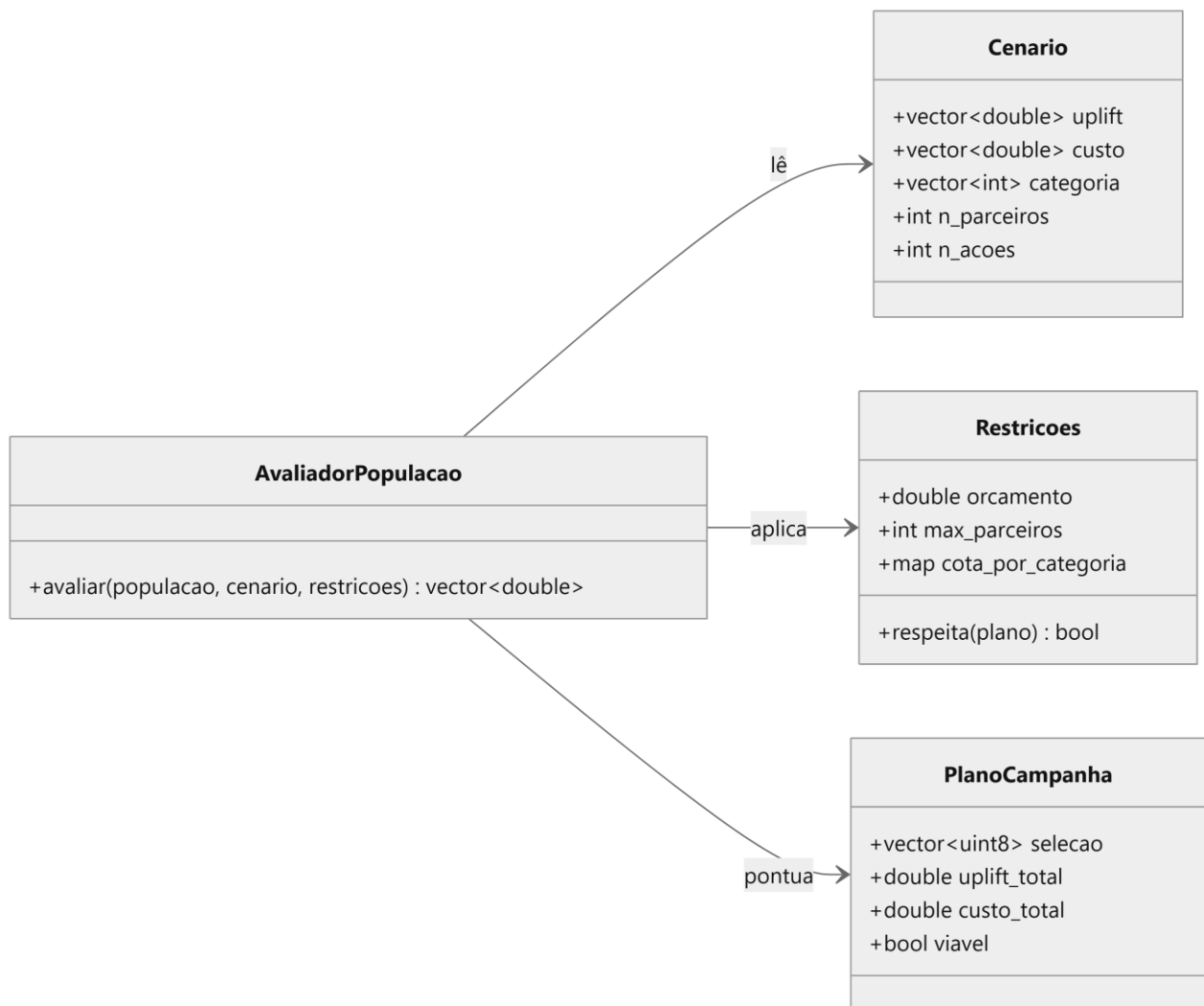


Figura 8 — Estruturas do núcleo em C++: cenário, restrições, plano e avaliador de população.

## 2.8 Núcleo computacional — as três implementações

Três implementações da mesma interface, para que o benchmark compare o que é comparável.

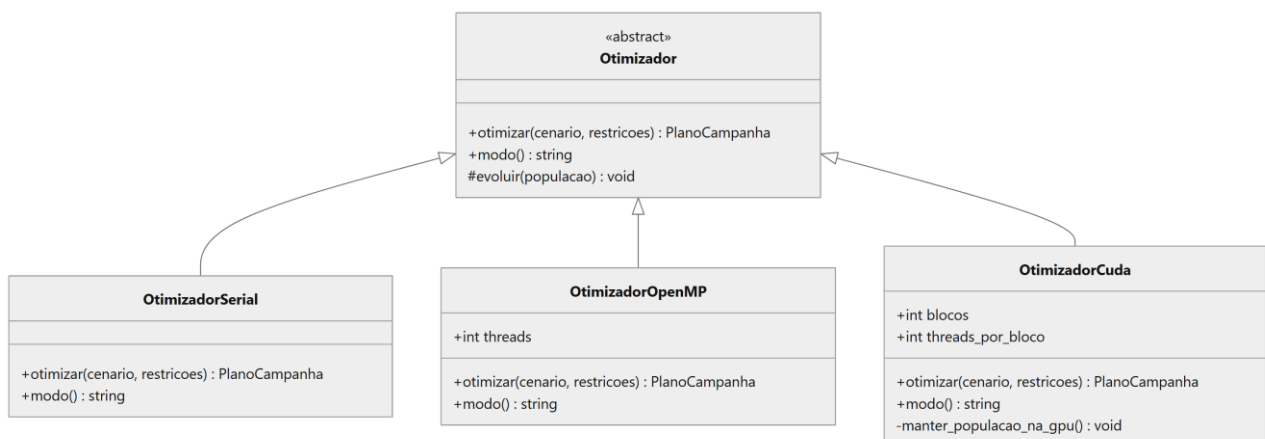


Figura 9 — Hierarquia do otimizador: serial, paralelo em CPU com OpenMP e paralelo em GPU com CUDA.

O método privado de manter a população na GPU não é detalhe de implementação: é o que justifica a classe existir. Sem ele, a GPU ganha apenas 1,1x da CPU paralela; com ele, 11x.

## 2.9 O que já está implementado

Um diagrama que promete código inexistente é pior que diagrama nenhum, porque ninguém reconfere. A separação abaixo pode ser verificada no repositório.

| Camada          | Implementado                                                                           | Previsto                                     |
|-----------------|----------------------------------------------------------------------------------------|----------------------------------------------|
| <b>Domínio</b>  | As 16 entidades, com restrições no banco                                               | —                                            |
| <b>Serviços</b> | Segurança, sessões, bloqueio, auditoria, dependências, leitor de relatório, importação | Segmentador, ranking, otimização, assistente |
| <b>Rotas</b>    | Sessão, usuários, importações, auditoria, saúde                                        | Painel, otimização, mensagens, assistente    |
| <b>Núcleo</b>   | Kernel de avaliação validado em CUDA e OpenMP                                          | Otimizador e as três implementações          |
| <b>Modelo</b>   | —                                                                                      | Previsor em PyTorch                          |

Cobertura de teste da parte implementada: **179 testes automatizados, 96% das linhas.**

### 3. Modelo Entidade-Relacionamento

O modelo conceitual responde "o que o sistema precisa saber", na linguagem do negócio e sem decisão de implementação. São dezesseis entidades.

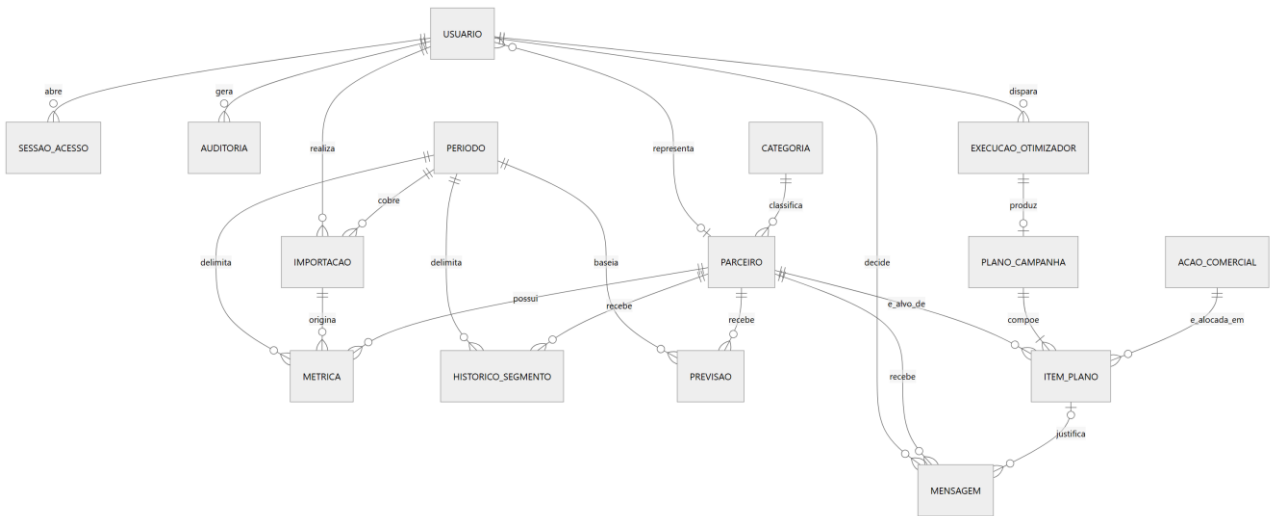


Figura 10 — Modelo conceitual: entidades e relacionamentos, com cardinalidade em notação pé-de-galinha.

A entidade de tentativas de login não aparece no diagrama porque não se relaciona com nenhuma outra, e isso é deliberado: ela registra tentativas inclusive contra logins inexistentes, e ligá-la ao usuário quebraria justamente o caso que ela existe para cobrir.

#### 3.1 Entidades e atributos

Os atributos estão aqui, e não dentro das caixas do desenho: com dezesseis entidades e mais de cem atributos, a figura ficaria ilegível impressa.

| Entidade           | O que representa             | Atributos                                                                 |
|--------------------|------------------------------|---------------------------------------------------------------------------|
| Usuario            | Quem acessa o sistema        | login, nome, senha (só hash), perfil, situação, data de cadastro          |
| SessaoAcesso       | Sessão autenticada em curso  | identificador, início, expiração, revogação e motivo, origem, navegador   |
| TentativaLogin     | Tentativa de autenticação    | login tentado, origem, resultado, momento                                 |
| Auditoria          | Ação sensível praticada      | autor, ação, parâmetros, origem, momento                                  |
| Categoria          | Ramo de atuação              | nome, situação                                                            |
| Parceiro           | Comércio da rede             | nome, categoria, origem da categoria, status comercial, contato, situação |
| Periodo            | Janela de tempo do relatório | data inicial, data final                                                  |
| Importacao         | Uma carga de relatório       | período, autor, origem, total gravado, total rejeitado, momento           |
| Metrica            | Desempenho num período       | faturamento, número de pedidos, projeção                                  |
| HistoricoSegmento  | Classificação num período    | segmento, momento do cálculo                                              |
| Previsao           | Estimativa do modelo         | faturamento previsto, probabilidade de queda, versão do modelo            |
| AcaoComercial      | Tipo de ação da campanha     | nome, custo unitário, uplift esperado, situação                           |
| ExecucaoOtimizador | Uma rodada do otimizador     | modo, parâmetros, viabilidade, restrição violada, uplift, custo, tempo    |
| PlanoCampanha      | Plano de uma execução viável | janela de aplicação                                                       |
| ItemPlano          | Par parceiro–ação escolhido  | uplift esperado, custo                                                    |
| Mensagem           | Comunicação para um parceiro | texto gerado, texto final, estado, autor da decisão, motivo               |

### 3.2 Relacionamentos que carregam regra de negócio

- Métrica é única por parceiro e período — é o que impede uma reimportação de duplicar a série e corromper a segmentação por tendência.
- Execução do otimizador produz zero ou um plano: ou o plano respeita todas as restrições, ou não existe plano.
- Plano de campanha compõe um ou mais itens, no máximo um por parceiro. Item sem plano não tem significado.
- Usuário representa zero ou um parceiro. Só o perfil Parceiro usa esse vínculo, e é ele que restringe o acesso ao próprio desempenho.

## 4. Modelo relacional

A conversão do modelo conceitual em tabelas. Toda tabela tem chave primária inteira e sequencial: a escolha por chave substituta mantém as chaves estrangeiras com uma coluna só, e permite corrigir o nome de um parceiro sem reescrever as métricas dele.

*Notação: o sublinhado marca chave primária e o asterisco marca chave estrangeira.*

```
usuario(id, login, nome, senha_hash, perfil, ativo, parceiro_id*, criado_em)
sessao_acesso(id, token_hash, usuario_id*, criada_em, expira_em, revogada_em,
               motivo_revogacao, origem, agente)
tentativa_login(id, login, origem, sucesso, ocorrido_em)
auditoria(id, usuario_id*, acao, detalhes, origem, ocorrido_em)

categoria(id, nome, ativa)
parceiro(id, nome, categoria_id*, origem_categoria, status, contato, ativo, criado_em)

periodo(id, data_inicio, data_fim)
importacao(id, periodo_id*, usuario_id*, origem, total_gravado, total_rejeitado, enviado_em)
metrica(id, parceiro_id*, periodo_id*, importacao_id*, faturamento, pedidos, projecao)
historico_segmento(id, parceiro_id*, periodo_id*, segmento, calculado_em)

previsao(id, parceiro_id*, periodo_base_id*, faturamento_previsto, probabilidade_queda,
          modelo_versao, gerada_em)
acao_comercial(id, nome, custo_unitario, uplift_esperado_pct, ativa)
execucao_otimizador(id, usuario_id*, modo, parametros, viavel, restricao_violada,
                    uplift_total, custo_total, tempo_ms, executada_em)
plano_campanha(id, execucao_id*, aplicacao_inicio, aplicacao_fim)
item_plano(id, plano_id*, parceiro_id*, acao_id*, uplift_esperado, custo)

mensagem(id, parceiro_id*, item_plano_id*, texto_gerado, texto_final, estado,
          decidida_por_id*, motivo_rejeicao, gerada_em, decidida_em)
```

### 4.1 Detalhamento das tabelas centrais

As demais tabelas seguem o mesmo padrão e estão detalhadas coluna a coluna na documentação do repositório.

#### usuario

| Coluna      | Tipo         | Chave         | Restrição                                 |
|-------------|--------------|---------------|-------------------------------------------|
| id          | serial       | PK            |                                           |
| login       | varchar(60)  |               | único                                     |
| nome        | varchar(120) |               |                                           |
| senha_hash  | varchar(255) |               | somente hash Argon2id                     |
| perfil      | enum         |               | ADMINISTRADOR, GESTOR, ANALISTA, PARCEIRO |
| ativo       | boolean      |               | padrão verdadeiro                         |
| parceiro_id | integer      | FK → parceiro | nulo, exceto no perfil PARCEIRO           |
| criado_em   | timestampz   |               | padrão now()                              |

Uma restrição do banco garante que ter vínculo com parceiro e ser do perfil Parceiro são a mesma condição, nos dois sentidos.

#### metrica — a tabela central do sistema

| Coluna | Tipo   | Chave | Restrição |
|--------|--------|-------|-----------|
| id     | serial | PK    |           |

| Coluna        | Tipo          | Chave           | Restrição                        |
|---------------|---------------|-----------------|----------------------------------|
| parceiro_id   | integer       | FK → parceiro   | único em conjunto com periodo_id |
| periodo_id    | integer       | FK → periodo    |                                  |
| importacao_id | integer       | FK → importacao | de qual carga o registro veio    |
| faturamento   | numeric(12,2) |                 | maior ou igual a zero            |
| pedidos       | integer       |                 | maior ou igual a zero            |
| projecao      | numeric(12,2) |                 | nulo permitido                   |

**Não existe coluna de ticket médio.** Ele é faturamento dividido por pedidos, calculado na consulta. Como coluna, divergiria das parcelas que o originam na primeira correção de dado.

## 4.2 O que o banco garante sozinho

Nove restrições impedem estado inválido independentemente do código da aplicação. É a diferença entre uma regra que vale e uma regra que valeria se ninguém esquecesse de chamá-la.

| Restrição                       | Garante                                                                           |
|---------------------------------|-----------------------------------------------------------------------------------|
| usuario · vínculo com parceiro  | Vínculo se e somente se o perfil for PARCEIRO                                     |
| parceiro · categoria com origem | Categoria sempre acompanhada da procedência                                       |
| periodo · ordem                 | Data final nunca anterior à inicial                                               |
| metrica · faturamento e pedidos | Nenhum dos dois negativo                                                          |
| previsao · probabilidade        | Probabilidade entre zero e um                                                     |
| acao_comercial · custo          | Custo de ação não negativo                                                        |
| execucao_otimizador · motivo    | Execução inviável tem o motivo registrado                                         |
| mensagem · autor da decisão     | Mensagem decidida tem autor e data — nenhuma sai do estado pendente sem um humano |

## 4.3 Índices

Cada índice existe por causa de uma consulta concreta, e não por precaução.

| Índice               | Colunas                 | Serve a                                      |
|----------------------|-------------------------|----------------------------------------------|
| parceiro por nome    | nome                    | Busca no painel e casamento na importação    |
| métrica por período  | periodo_id, faturamento | Ranking do período sem varrer a tabela       |
| segmento por período | periodo_id, segmento    | Filtro por segmento no painel                |
| auditoria por data   | ocorrido_em             | Consulta da trilha por intervalo             |
| sessão por usuário   | usuario_id              | Derrubar as sessões de um usuário de uma vez |
| tentativa por origem | origem, ocorrido_em     | Contagem de falhas na janela do bloqueio     |
| mensagem por estado  | estado                  | Fila de aprovação                            |

## 5. Protótipo das telas principais

Quatro telas navegáveis, construídas em HTML e CSS autorais. O protótipo não é uma imagem estática: é uma página que se abre no navegador e permite percorrer as telas, e está versionada no repositório junto com o restante do projeto.

Protótipo navegável: [github.com/PedroMiranda243/gih-fabrica-de-software/blob/main/docs/prototipo/index.html](https://github.com/PedroMiranda243/gih-fabrica-de-software/blob/main/docs/prototipo/index.html)

**Todos os dados exibidos são sintéticos**, gerados por script. Nenhum nome de empresa, pessoa ou operação real aparece no protótipo ou em qualquer parte do repositório, que é público.

### 5.1 Decisões visuais que valem explicar

- Cada cor tem um trabalho só. O âmbar significa "responde ao seu clique" — botão, link, foco — e nada mais. Quando uma cor é marca, ação e dado ao mesmo tempo, ela deixa de significar qualquer coisa.
- Cada segmento tem uma cor e só uma, idêntica em tabela, gráfico e indicador.
- A paleta foi validada com ferramenta, não escolhida no olho: o primeiro conjunto reprovou porque vermelho e verde ficavam indistinguíveis para daltônicos, e o verde foi trocado por turquesa.
- Nenhuma informação depende só da cor: o rótulo do segmento aparece sempre em texto ao lado do ponto colorido.

5.2 Painel

Responde à pergunta dominante do produto: quem precisa de atenção agora. Indicadores no topo, série histórica e distribuição por segmento, e o ranking de parceiros abaixo.

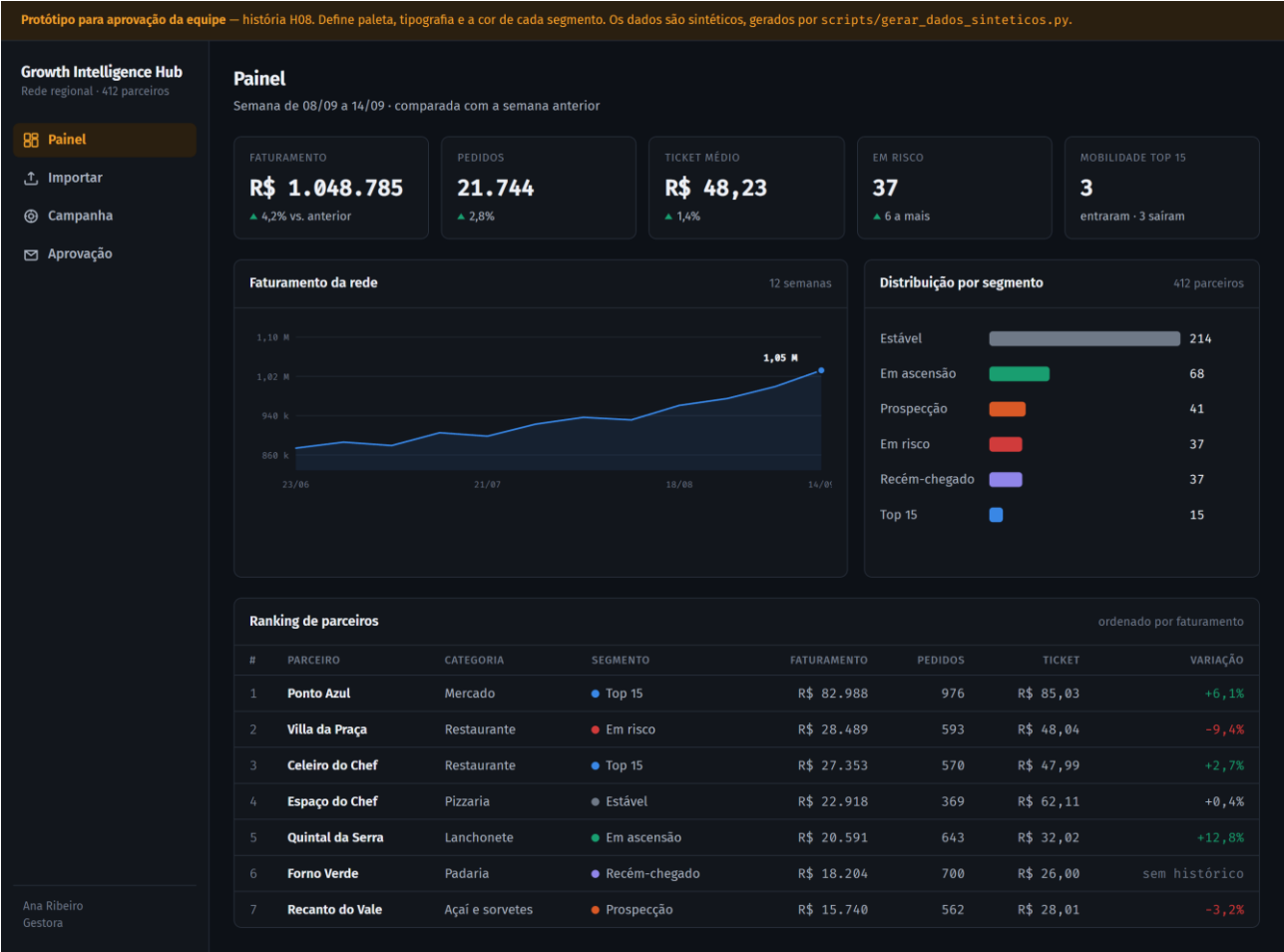


Figura 11 — Tela de painel: indicadores consolidados, faturamento da rede, distribuição por segmento e ranking.

### 5.3 Importação

O período é campo obrigatório e não tem valor padrão. A prévia mostra o que será gravado — reconhecidos, rejeitados com o motivo de cada um, e parceiros ainda não cadastrados — antes de qualquer gravação.

Protótipo para aprovação da equipe — história H08. Define paleta, tipografia e a cor de cada segmento. Os dados são sintéticos, gerados por scripts/gerar\_dados\_sinteticos.py.

Growth Intelligence Hub

Rede regional · 412 parceiros

Painel

Importar

Campanha

Aprovação

Importar relatório

Cole o texto do relatório ou envie um arquivo CSV

Período

obrigatório

Data inicial \*

08 / 09 / 2026

Data final \*

14 / 09 / 2026

O relatório de origem não traz datas. Sem o período, as métricas ficam sem lugar na linha do tempo e a segmentação por tendência passa a classificar errado sem avisar.

Prévia

412 reconhecidos · 3 rejeitados

| PARCEIRO       | FATURAMENTO   | PEDIDOS | SITUAÇÃO               |
|----------------|---------------|---------|------------------------|
| Ponto Azul     | R\$ 82.988,29 | 976     | ● Reconhecido          |
| Villa da Praça | R\$ 28.489,35 | 593     | ● Reconhecido          |
| Forno Dourado  | R\$ 4.102,00  | 158     | ● Parceiro novo        |
| Parada Central | —             | —       | ● Faturamento ilegível |

Confirmar importação

Cancelar

⚠ Este período já foi importado em 09/09 por Ana Ribeiro, com 409 registros. Substituir apaga os dados anteriores. O padrão é cancelar.

Ana Ribeiro

Gestora

Figura 12 — Tela de importação: período obrigatório e prévia com reconhecidos, rejeitados e parceiros novos.

Growth Intelligence Hub · Grupo 18 · Sprint 03 · 42

## 5.4 Campanha

Onde o núcleo computacional aparece para o usuário: restrições de orçamento e cota, escolha do modo de execução, comparativo de desempenho entre serial, CPU paralela e GPU, e o plano recomendado.

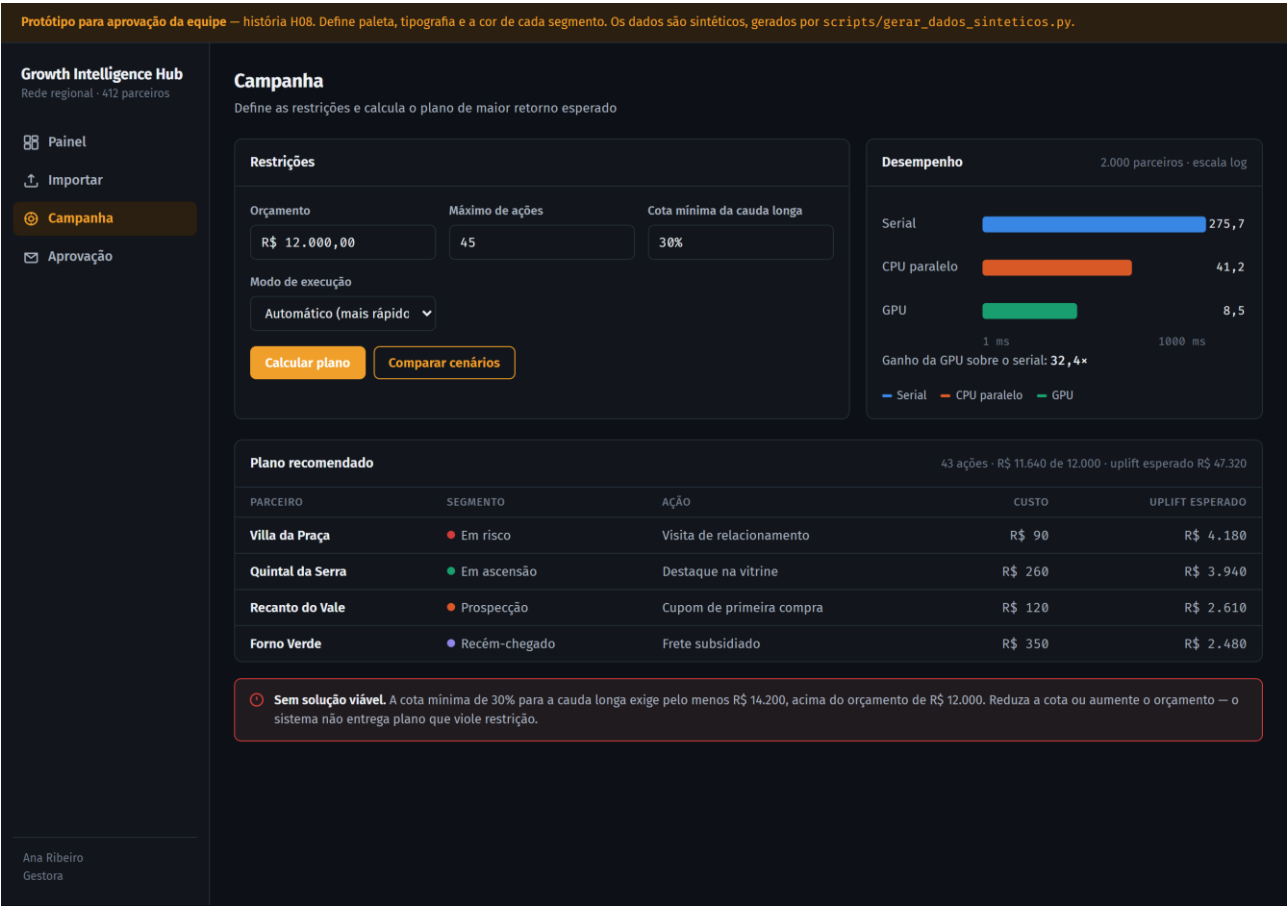


Figura 13 — Tela de campanha: restrições, comparativo de desempenho entre os três modos e plano recomendado.

## 5.5 Aprovação

Nenhuma mensagem chega ao parceiro sem decisão humana. A fila mostra cada mensagem com o destinatário, o segmento e o texto, e o gestor aprova, edita ou rejeita.

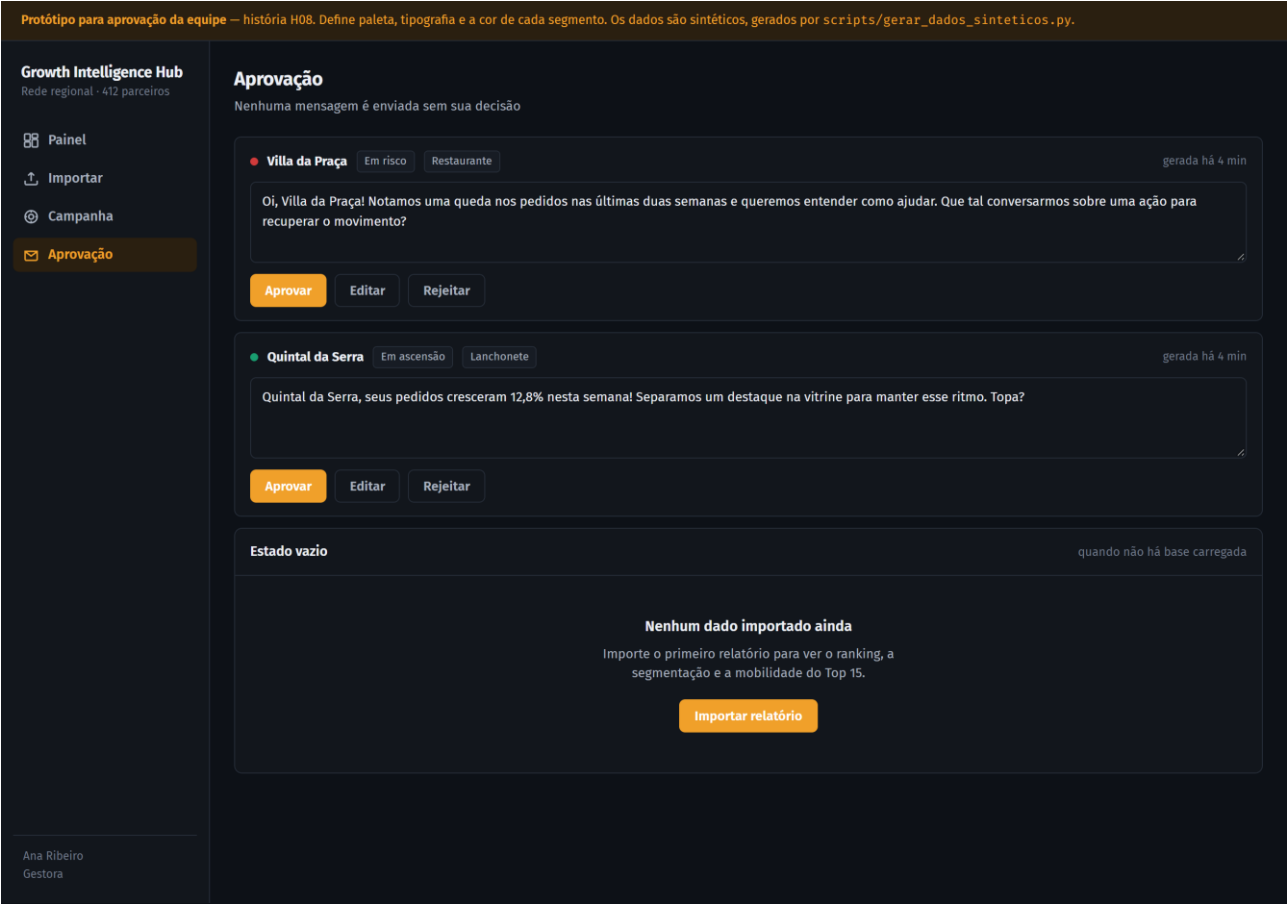


Figura 14 — Tela de aprovação: fila de mensagens pendentes, com a decisão humana como passo obrigatório.

## 6. Banco de dados criado

O esquema não está apenas desenhado: está aplicado e em uso. O ambiente completo sobe com um comando, e o processo de inicialização aplica as migrações antes de servir a API.

### 6.1 Estrutura aplicada

| Objeto                  | Quantidade |
|-------------------------|------------|
| Tabelas de domínio      | 16         |
| Chaves primárias        | 16         |
| Chaves estrangeiras     | 21         |
| Restrições de unicidade | 11         |
| Restrições de validação | 9          |
| Índices                 | 34         |
| Tipos enumerados        | 7          |

Consultado no catálogo do próprio PostgreSQL, e não estimado a partir do código:

```
$ docker compose exec api alembic current
77b3651bd03d (head)

$ docker compose exec postgres psql -U gih -d gih -c "\dt"
      acao_comercial      execucao_otimizador      item_plano      metrica      periodo
      alembic_version      historico_segmento      mensagem      parceiro      plano_campanha
      auditoria      importacao      previsao      categoria      sessao_acesso
      tentativa_login      usuario
(17 rows)
```

São as 16 tabelas de domínio mais a tabela de controle da ferramenta de migração, que registra qual versão do esquema está aplicada.

### 6.2 Esquema versionado, não script solto

O banco é criado por migrações versionadas no repositório. Isso permite que qualquer integrante chegue ao mesmo estado a partir de um clone limpo, e torna a evolução do banco auditável no histórico. O ciclo de reverter e reaplicar foi testado duas vezes, e não uma: a falha conhecida de tipos enumerados só aparece na segunda execução.

A suíte de testes também aplica as mesmas migrações num banco separado, em vez de montar o esquema por outro caminho. Custa alguns segundos por execução e cobre a divergência entre o modelo em código e a migração — defeito que ninguém percebe até a hora de implantar.

## 7. Projeto estruturado no GitHub

Repositório: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

### 7.1 Organização

| Diretório | Conteúdo                                                                      |
|-----------|-------------------------------------------------------------------------------|
| api/      | API em Python e FastAPI, migrações do banco e suíte de testes                 |
| nucleo/   | Núcleo computacional em C++ e CUDA, com a investigação técnica já concluída   |
| docs/     | Toda a documentação técnica, diagramas, protótipo e o gerador deste documento |
| scripts/  | Gerador de dados sintéticos e comando de reinicialização do banco             |
| .github/  | Integração contínua e mapa de responsáveis por revisão                        |
| web/      | Interface: login, painel, importação e parceiros (construída na Sprint 03)    |

### 7.2 Evidências de evolução

A disciplina pede que o repositório demonstre a evolução do projeto. O que está lá para ser verificado:

- Histórias de usuário registradas como issues, agrupadas por sprint em milestones, com critérios de aceite.
- Toda alteração entra por Pull Request revisado. A branch principal é protegida e não aceita push direto.
- Integração contínua obrigatória: análise estática, suíte de testes contra um PostgreSQL real, e verificação de que nenhum dado ou segredo foi versionado.
- Mapa de responsáveis por diretório, que convida automaticamente o dono de cada área para revisar.
- Documentação versionada junto com o código, atualizada no mesmo Pull Request que introduz a mudança.

### 7.3 Estado atual do desenvolvimento

| Entrega                            | Situação       | Evidência                                                            |
|------------------------------------|----------------|----------------------------------------------------------------------|
| Banco modelado e migrado           | Concluído      | 16 tabelas, 21 chaves estrangeiras, 9 restrições                     |
| Ambiente sobe com um comando       | Concluído      | Docker Compose com migração automática                               |
| Autenticação, perfis e auditoria   | Concluído      | Testes cobrindo cada endpoint contra cada perfil                     |
| Importação de relatório com prévia | Concluído      | Interpretador tolerante, com testes próprios                         |
| Núcleo em GPU                      | Risco retirado | Ganho medido de até 100x sobre o baseline serial                     |
| Protótipo das telas                | Aprovado       | Aprovado pela equipe em 17/09/2026 — quatro telas, paleta validada   |
| Interface em React                 | Concluído      | Construída na Sprint 03 sobre o protótipo aprovado — ver a Parte III |

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*

## Parte III — Sprint 03: Estrutura Inicial Funcionando

Esta parte demonstra o sistema em funcionamento. Os seis itens da entrega estão implementados, persistem no banco e são exercitados por teste automatizado — e cada um aparece aqui com saída de execução real, não com descrição.

O sistema já tem interface. Depois que a equipe aprovou o protótipo das telas, a aplicação web foi construída sobre o sistema visual validado: login, painel, importação de relatório e gestão de parceiros, todas consumindo a API real sobre o banco real. As capturas de tela desta parte são da aplicação no ar, e não do protótipo — e cada uma vem acompanhada da execução que a sustenta.

Uma lacuna está declarada em vez de omitida: o cadastro de usuários funciona pela API e por um comando de terminal, mas ainda não tem tela. A seção 4 diz isso com essas palavras.

### Os seis itens, e onde estão

| # | Item                     | Situação    | Evidência neste documento                                       |
|---|--------------------------|-------------|-----------------------------------------------------------------|
| 1 | Banco de dados conectado | Funcionando | Seção 2 — verificação, catálogo do banco e importação pela tela |
| 2 | Login funcional          | Funcionando | Seção 3 — tela de login e transcrição das requisições           |
| 3 | Cadastro de usuários     | Funcionando | Seção 4 — transcrição, persistência e comando de terminal       |
| 4 | Controle de perfis       | Funcionando | Seção 5 — negação por perfil                                    |
| 5 | CRUD principal           | Funcionando | Seção 6 — as quatro operações e a tela de parceiros             |
| 6 | Deploy local             | Funcionando | Seção 7 — um comando sobe banco, API e interface                |

## 1. A estrutura implementada

O sistema está organizado em camadas com responsabilidades separadas, e a separação é o que permite medir o núcleo computacional isoladamente e garantir que a regra de negócio não escape para a interface.

### 1.1 O que existe hoje

| Camada      | O que está implementado                                                                                          | Como se verifica                              |
|-------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Interface   | Login, painel, importação de relatório e gestão de parceiros, sobre o sistema visual aprovado                    | Testes da interface e as capturas desta parte |
| Banco       | 16 tabelas, 21 chaves estrangeiras, 9 restrições de validação, 35 índices                                        | Migração versionada, aplicada na subida       |
| Acesso      | Sessão com estado no servidor, senha em Argon2id, bloqueio por força bruta                                       | 48 verificações contra a API no ar            |
| Usuários    | Cadastro, edição, desativação e atribuição de perfil, pela API e por comando de terminal                         | Testes automatizados e transcrição            |
| Autorização | Quatro perfis, verificados no servidor a cada requisição                                                         | Cada endpoint contra cada perfil              |
| Parceiros   | CRUD completo, com busca sem diferenciar acento, filtros e exclusão condicional                                  | Transcrição das quatro operações e a tela     |
| Ingestão    | Importação por texto ou arquivo CSV, com prévia, período obrigatório, rejeição parcial e substituição de período | Verificação de ponta a ponta e a tela         |
| Painel      | Indicadores do período, ranking com variação de posição e série histórica                                        | Testes automatizados e a tela                 |
| Auditoria   | Trilha de ações sensíveis, com filtro e paginação                                                                | Consulta na própria API                       |
| Núcleo      | Kernel de avaliação validado em CUDA e OpenMP                                                                    | Medição registrada no repositório             |

### 1.2 Números do que foi construído

Medidos nesta entrega, contra o ambiente no ar — nenhum é estimado:

| Medida                              | Valor |
|-------------------------------------|-------|
| Operações da API em funcionamento   | 26    |
| Testes automatizados da API         | 348   |
| Testes automatizados da interface   | 15    |
| Cobertura de linhas da API          | 97%   |
| Verificações de ponta a ponta       | 48    |
| Tabelas no banco                    | 16    |
| Decisões de arquitetura registradas | 9     |

### 1.3 Como a qualidade é verificada

- Toda alteração entra por Pull Request revisado; a branch principal é protegida.
- A integração contínua roda análise estática e a suíte completa contra um PostgreSQL real a cada Pull Request.
- A suíte de testes aplica as mesmas migrações do ambiente real, em um banco separado — e não monta o esquema por outro caminho, para cobrir a divergência entre o modelo em código e a migração.
- Um teste percorre cada endpoint contra cada perfil. Endpoint novo sem permissão declarada reprova a suíte, o que impede a cobertura de envelhecer em silêncio.
- O contraste das cores da interface é medido pintando cada cor e lendo o pixel, nos modos claro e escuro — e não suposto a partir da paleta.

### 1.4 A interface em funcionamento

O painel é a primeira tela depois do login. Resume o período mais recente, mostra a tendência da rede e ordena os parceiros por faturamento, com a posição anterior de cada um. Nenhum número desta tela é calculado no navegador: variação, ticket médio e posição vêm prontos da API, e a interface só formata e desenha.

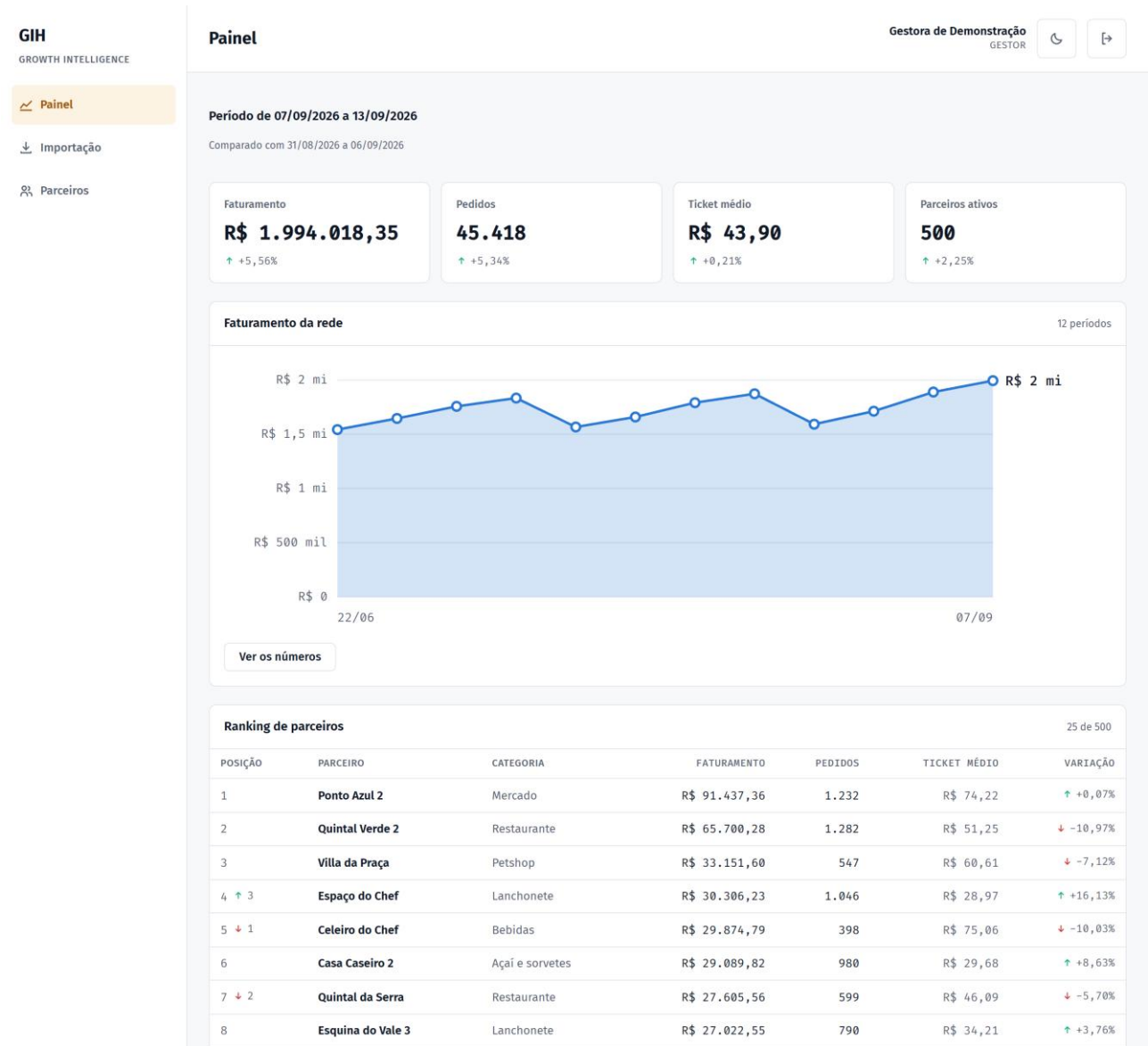


Figura 15 — Painel com os dados do banco: indicadores do período com a variação, série histórica e ranking.

## 2. Banco de dados conectado

A aplicação conversa com o PostgreSQL desde a primeira requisição. A conexão é verificada em três níveis: a própria API reporta o estado do banco, a migração aplicada é identificável, e a trilha de auditoria só tem conteúdo porque houve escrita real.

### 2.1 Verificação executada

A saída abaixo é de uma execução real do comando de verificação, que roda contra a API no ar — HTTP real, cookie real, banco real:

```
[1/6] Banco de dados conectado
      ok   a API responde à verificação de saúde
      ok   a API confirma o banco no ar  banco='ok'
      ok   o banco tem registros gravados pela aplicação  315 eventos na trilha
```

### 2.2 Estrutura aplicada

Consultado no catálogo do próprio PostgreSQL, e não estimado a partir do código:

```
$ docker compose exec api alembic current
a41f9c2be107 (head)

$ docker compose exec postgres psql -U gih -d gih -c "\dt"
      acao_comercial      execucao_otimizador      item_plano      metrica      periodo
      alembic_version      historico_segmento      mensagem      parceiro      plano_campanha
      auditoria      importacao      previsao      categoria      sessao_acesso
      tentativa_login      usuario
(17 rows)
```

São as 16 tabelas de domínio mais a tabela de controle da ferramenta de migração. O esquema é criado por migração versionada no repositório, o que permite a qualquer integrante chegar ao mesmo estado a partir de um clone limpo.

### 2.3 O dado entrando pela interface

A importação é por onde o banco é alimentado, e a tela segue o mesmo fluxo que a API exige: período obrigatório, prévia antes de gravar, e a rejeição de cada linha com o motivo. Na captura, a linha com faturamento "abacaxi" é recusada e dita — e as outras três seguem para gravação.

GIH  
GROWTH INTELLIGENCE

Painel

Importação

Parceiros

Importação de relatório

Gestora de Demonstração  
GESTOR

Relatório do período

Colar o texto ou enviar um arquivo CSV

Início do período

14 / 09 / 2026

Fim do período

20 / 09 / 2026

O relatório não traz datas. Sem o período, as métricas ficam órfãs na linha do tempo e a segmentação por tendência classifica errado sem emitir erro — por isso ele é obrigatório.

Colar o relatório

Parceiro;Faturamento;Pedidos  
Casa da Praça;4210,55;98  
Espaço Azul;7730,10;140  
Empório Bom Prato;abacaxi;38  
Quiosque do Porto;1890,00;52

...ou enviar um arquivo

Escolher arquivo

Nenhum arquivo escolhido

Ver a prévia

Prévia

3 reconhecidas · 1 rejeitadas

1 parceiro(s) novo(s) serão cadastrados: Quiosque do Porto

| LINHA             | CONTEÚDO                     | MOTIVO                               |
|-------------------|------------------------------|--------------------------------------|
| 4                 | Empório Bom Prato;abacaxi;38 | Faturamento não numérico: 'abacaxi'. |
| PARCEIRO          | FATURAMENTO                  | PEDIDOS                              |
| Casa da Praça     | R\$ 4.210,55                 | 98                                   |
| Espaço Azul       | R\$ 7.730,10                 | 140                                  |
| Quiosque do Porto | R\$ 1.890,00                 | 52                                   |

Gravar 3 registros

Figura 16 — Importação com a prévia aberta: três linhas reconhecidas, uma rejeitada com o motivo e um parceiro novo.

### 3. Login funcional

A autenticação é por login e senha, integrada ao banco. A sessão tem estado no servidor: encerrar a sessão a invalida de verdade, e não apenas apaga o cookie do navegador.

#### 3.1 Como funciona

- A senha é armazenada apenas como hash Argon2id, com sal por usuário. Nenhuma senha em texto claro no banco ou em log.
- O identificador da sessão são 256 bits sorteados, guardados no banco como hash — quem ler a tabela não consegue se passar por ninguém.
- O cookie vem com HttpOnly e SameSite, e o identificador é renovado a cada autenticação, o que impede fixação de sessão.
- Cinco tentativas falhas a partir da mesma origem bloqueiam temporariamente novas tentativas.
- Login inexistente e senha errada respondem de forma idêntica — mesma mensagem, mesmo código e mesmo tempo. Responder mais rápido para um login que não existe entregaria a lista de usuários válidos.

#### 3.2 Verificação executada

```
[2/6] Login funcional
ok  credencial válida abre sessão
ok  o cookie de sessão vem protegido  HttpOnly presente
ok  a resposta não devolve senha
ok  a sessão identifica o usuário
ok  encerrar invalida a sessão no servidor
ok  senha errada é recusada
ok  usuário inexistente responde igual a senha errada  indistinguível, como pede o RNF11
```

#### 3.3 A requisição e a resposta

Transcrição de uma autenticação real. A senha aparece mascarada de propósito:

Login – item 2 da entrega

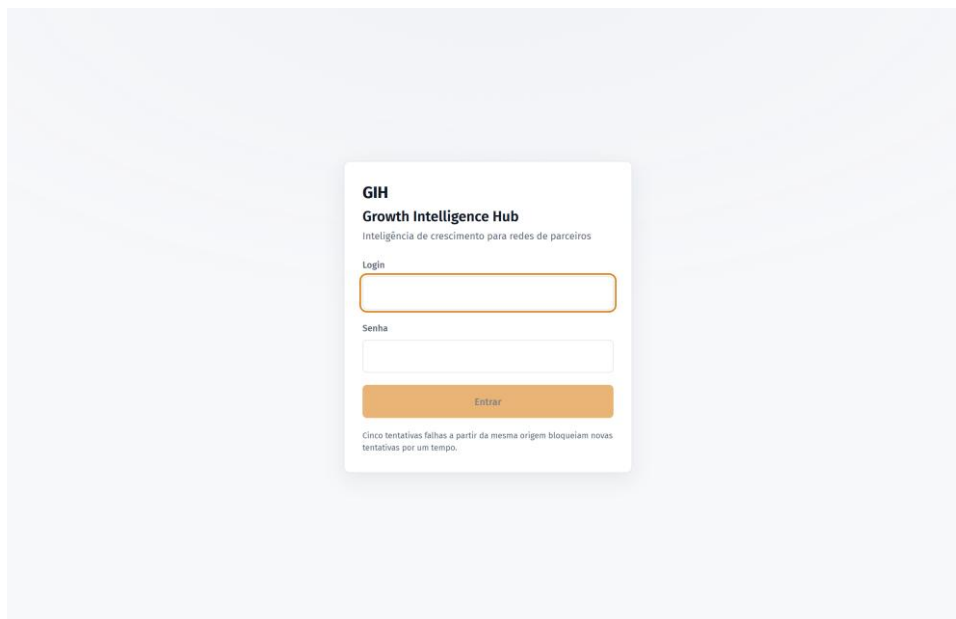
```
=====

$ POST /api/sessao
> {
>   "login": "t94926.analista",
>   "senha": "*****"
> }
201 Created
< {
<   "usuario": {
<     "id": 28,
<     "login": "t94926.analista",
<     "nome": "Analista da Evidência",
<     "perfil": "ANALISTA",
<     "ativo": true,
<     "parceiro_id": null,
<     "criado_em": "2026-09-19T01:08:26.642762Z"
<   },
<   "expira_em": "2026-09-19T09:08:29.040466Z"
< }

=====
```

### 3.4 A tela de login

A tela não distingue login inexistente de senha errada — a API responde igual aos dois, e a interface mostra a mensagem que recebeu em vez de tentar ser prestativa. O contorno em destaque é o foco do teclado, visível em todo controle.



**Figura 17** — Tela de login da aplicação, com o foco no primeiro campo.

## 4. Cadastro de usuários

O cadastro é do Administrador, persiste no banco e registra a criação na trilha de auditoria. Cada usuário recebe exatamente um perfil, entre Administrador, Gestor, Analista e Parceiro.

### 4.1 Verificação executada

```
[3/6] Cadastro de usuários
ok   cria usuário com perfil ADMINISTRADOR e2e25564.administrador
ok   cria usuário com perfil GESTOR e2e25564.gestor
ok   cria usuário com perfil ANALISTA e2e25564.analista
ok   os usuários criados estão persistidos
ok   a listagem não expõe hash de senha
```

### 4.2 A requisição e a resposta

Preparação — um analista, que é quem gerencia parceiros (UC04)

```
=====
$ POST /api/usuarios
> {
>   "login": "t94926.analista",
>   "nome": "Analista da Evidência",
>   "senha": "*****",
>   "perfil": "ANALISTA"
> }
201 Created
< {
<   "id": 28,
<   "login": "t94926.analista",
<   "nome": "Analista da Evidência",
<   "perfil": "ANALISTA",
<   "ativo": true,
<   "parceiro_id": null,
<   "criado_em": "2026-09-19T01:08:26.642762Z"
< }
=====
```

**A resposta não devolve a senha nem o hash dela.** As respostas da API são declaradas campo a campo, e não serializam a entidade inteira — é o que impede um campo sensível de vaziar quando alguém acrescentar uma coluna nova.

### 4.3 Pelo terminal

O cadastro também existe como comando, e por necessidade: a senha do administrador inicial é sorteada e só aparece no log da primeira subida, e importação e parceiros nem são do Administrador. Sem um caminho reproduzível, quem clonasse o projeto pararia na tela de login.

```
$ docker compose exec api python -m app.cli criar-usuario \
  --login seu.login --nome "Seu Nome" --perfil GESTOR
Senha para seu.login:
Repita a senha:
Usuário criado: seu.login (GESTOR)
```

A senha é digitada no terminal, nunca passada como argumento — argumento fica no histórico do shell. O comando aplica as mesmas validações da API e registra a criação na trilha de auditoria. Há também um comando para redefinir a senha, que encerra as sessões abertas daquele usuário.

**O que ainda não existe:** a tela de administração de usuários. O cadastro funciona, persiste e é verificado — pela API e pelo terminal —, mas a interface ainda não o oferece. Está entre os próximos passos.

### 4.4 Não existe apagar usuário

A trilha de auditoria referencia o autor de cada ação, e remover a linha deixaria o histórico apontando para o nada. O que existe é desativar, que resolve o problema real — tirar o acesso — sem destruir o registro. Desativar também encerra as sessões abertas daquele usuário na hora.

Há ainda uma trava: a última conta de Administrador ativa não pode ser desativada nem rebaixada. Sem ela, uma edição distraída deixaria o sistema sem ninguém capaz de gerenciar usuários.

## 5. Controle de perfis

A autorização é verificada no servidor a cada requisição. Esconder um botão não é controle de acesso: mesmo que a interface não ofereça o caminho, o endereço continua acessível por outros meios, e é o servidor que precisa recusar.

### 5.1 A matriz

Os perfis e o que cada um alcança vêm da matriz de permissões definida na Sprint 01, e não de leitura do código:

| Área                | Administrador | Gestor | Analista | Parceiro |
|---------------------|---------------|--------|----------|----------|
| Autenticar          | sim           | sim    | sim      | sim      |
| Gerenciar usuários  | sim           | —      | —        | —        |
| Auditar ações       | sim           | —      | —        | —        |
| Gerenciar parceiros | —             | sim    | sim      | —        |
| Importar relatório  | —             | sim    | sim      | —        |
| Painel e ranking    | só leitura    | sim    | sim      | —        |

### 5.2 Verificação executada

Cada rota protegida foi exercitada com cada perfil, e o resultado confrontado com a matriz:

```
[4/6] Controle de perfis
  · 25 rotas conferidas contra 3 perfis, a partir da matriz de docs/03-casos-de-uso.md
ok sem sessão, todo endpoint protegido responde 401
ok perfil ADMINISTRADOR só acessa o que a matriz permite
ok perfil GESTOR só acessa o que a matriz permite
ok perfil ANALISTA só acessa o que a matriz permite
```

### 5.3 A negação, na prática

Um analista tentando a área de usuários, que é exclusiva do Administrador:

Controle de perfis – item 4 da entrega

=====

O mesmo analista tentando a área de usuários, que é do Administrador:

```
$ GET /api/usuarios
403 Forbidden
< {
<   "detail": "Seu perfil não permite esta operação."
< }
```

=====

## 6. CRUD principal — parceiros

O parceiro é a entidade central do produto: métrica, segmento, previsão, plano de campanha e mensagem pendem dele. As quatro operações estão implementadas, integradas ao banco, e são exercitadas por 41 testes automatizados além da verificação de ponta a ponta.

### 6.1 As operações

| Operação  | Endereço                   | Comportamento                                                                                        |
|-----------|----------------------------|------------------------------------------------------------------------------------------------------|
| Cadastrar | POST /api/parceiros        | Nome único, garantido pelo banco; categoria escolhida vira confirmada                                |
| Consultar | GET /api/parceiros         | Busca parcial por nome sem diferenciar maiúscula nem acento; filtro por categoria, status e situação |
| Consultar | GET /api/parceiros/{id}    | Um parceiro, com a categoria resolvida                                                               |
| Atualizar | PATCH /api/parceiros/{id}  | Nome, categoria, status, contato e situação                                                          |
| Excluir   | DELETE /api/parceiros/{id} | Exclui sem histórico; recusa explicando quando há                                                    |

### 6.2 Verificação executada

```
[5/6] CRUD principal — parceiros
ok  cadastrar  Comércio e2e25564
ok  consultar
ok  consultar pela busca por nome
ok  cadastrar categoria
ok  atualizar
ok  a categoria confirmada por pessoa é gravada como MANUAL  RN05
ok  nome repetido é recusado
ok  excluir
ok  o excluído some da consulta
ok  excluir parceiro com histórico é recusado  409
ok  a recusa diz quantos registros impedem e aponta a desativação
```

## 6.3 Cadastrar, consultar e atualizar

Transcrição das requisições reais, com o corpo enviado e o corpo recebido:

CADASTRAR – item 5 da entrega

```
=====
$ POST /api/categorias
> {
>   "nome": "Categoria t94926"
> }
201 Created
< {
<   "id": 20,
<   "nome": "Categoria t94926",
<   "ativa": true
< }

$ POST /api/parceiros
> {
>   "nome": "Comércio t94926",
>   "status": "PROSPECCAO",
>   "contato": "contato@exemplo.test"
> }
201 Created
< {
<   "id": 549,
<   "nome": "Comércio t94926",
<   "categoria": null,
<   "origem_categoria": null,
<   "status": "PROSPECCAO",
<   "contato": "contato@exemplo.test",
<   "ativo": true,
<   "criado_em": "2026-09-19T01:08:29.087865Z"
< }

=====
```

ATUALIZAR

```
=====
$ PATCH /api/parceiros/549
> {
>   "status": "ATIVO",
>   "categoria_id": 20
> }
200 OK
< {
<   "id": 549,
<   "nome": "Comércio t94926",
<   "categoria": {
<     "id": 20,
<     "nome": "Categoria t94926",
<     "ativa": true
<   },
<   "origem_categoria": "MANUAL",
<   "status": "ATIVO",
<   "contato": "contato@exemplo.test",
<   "ativo": true,
<   "criado_em": "2026-09-19T01:08:29.087865Z"
< }

=====
```

## 6.4 A exclusão, e por que ela tem duas saídas

A entrega pede a operação de excluir. Excluir um parceiro que já tem faturamento importado, porém, falsearia as séries dos períodos já fechados: o total da rede deixaria de bater com a soma das partes, e ninguém perceberia, porque a tela continuaria parecendo correta.

A solução não foi escolher um dos dois lados. Sem histórico, o parceiro é apagado de verdade. Com histórico, a exclusão é recusada dizendo quantos registros de cada tipo impedem, e apontando a desativação — que resolve o problema real, que é tirar o parceiro de circulação.

EXCLUIR — sem histórico, a exclusão acontece

```
=====
$ DELETE /api/parceiros/549
204 No Content
< (sem conteúdo)

$ GET /api/parceiros/549
404 Not Found
< {
<   "detail": "Parceiro não encontrado."
< }
```

=====

E o caminho que protege o histórico:

```
$ DELETE /api/parceiros/549
204 No Content
< (sem conteúdo)
```

```
$ GET /api/parceiros/549
404 Not Found
< {
  < "detail": "Parceiro não encontrado."
  < }
```

=====

EXCLUIR – com histórico, a exclusão é recusada com o motivo

=====

Primeiro uma importação, para o parceiro passar a ter faturamento:

```
$ POST /api/importacoes
> {
  > "periodo_inicio": "2042-10-18",
  > "periodo_fim": "2042-10-24",
  > "texto": "Parceiro;Faturamento;Pedidos\nComércio t94926 hist;12500,40;312\n"
  > }
201 Created
< {
  < "id": 43,
  < "periodo": {
    < "id": 36,
    < "data_inicio": "2042-10-18",
    < "data_fim": "2042-10-24"
    < },
  < "autor": {
    < "id": 28,
    < "nome": "Analista da Evidência"
    < },
  < "origem": "TEXT0",
  < "total_gravado": 1,
  < "total_rejeitado": 0,
  < "enviado_em": "2026-09-19T01:08:29.169613Z",
  < "metricas_vigentes": 0
  < }
```

```
$ DELETE /api/parceiros/551
409 Conflict
< {
  < "detail": {
    < "erro": "O parceiro 'Comércio t94926 hist' tem histórico e não pode ser excluído.",
    < "ajuda": "Excluir apagar registros que outros períodos já contabilizam. Para tirá-lo
    de circulação sem perder o histórico, desative-o: PATCH neste mesmo endereço com
    {\\"ativo\": false}.",
    < "vinculos": {
      < "metricas": 1,
      < "segmentos": 0,
      < "previsoes": 0,
      < "itens_de_plano": 0,
      < "mensagens": 0,
      < "usuarios": 0
    < }
  < }
  < }
```

Apagar um parceiro com faturamento importado falsearia as séries dos períodos já fechados. A recusa diz o que impede e aponta a desativação.

```
$ PATCH /api/parceiros/551
> {
  > "ativo": false
  > }
200 OK
< {
  < "id": 551,
  < "nome": "Comércio t94926 hist",
  < "categoria": null,
```

```
<  "origem_categoria": null,  
<  "status": "ATIVO",  
<  "contato": null,  
<  "ativo": false,  
<  "criado_em": "2026-09-19T01:08:29.169613Z"  
< }
```

=====

### 6.5 Na interface

A consulta pela tela, com a busca "praca" encontrando os parceiros com "Praça" no nome. A busca ignora maiúscula e acento, e usa índice no banco — com 10.000 parceiros, que é a carga prevista, uma função aplicada linha a linha viraria varredura completa. Os filtros ficam no endereço da página, então um recorte pode ser mandado para outra pessoa por link.

GIH  
GROWTH INTELLIGENCE

Painel

Importação

Parceiros

Parceiros

Gestora de Demonstração  
GESTOR

Buscar por nome

praca

Categoria

Todas as categorias

Status comercial

Todos os status

Situação

Todas as situações

Parceiros

31 encontrados

| PARCEIRO           | CATEGORIA   | STATUS | SITUAÇÃO | CONTATO | CADASTRADO EM |
|--------------------|-------------|--------|----------|---------|---------------|
| Cantina da Praça   | Restaurante | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Casa da Praça      | Lanchonete  | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Casa da Praça 2    | Bebidas     | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Casa da Praça 3    | Gás e água  | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Celeiro da Praça   | Padaria     | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Celeiro da Praça 2 | Restaurante | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Empório da Praça   | Lanchonete  | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Empório da Praça 2 | Pizzaria    | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Empório da Praça 3 | Lanchonete  | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Espaço da Praça    | Farmácia    | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Espaço da Praça 3  | Restaurante | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Esquina da Praça   | Lanchonete  | ATIVO  | ● Ativo  | —       | 18/09/2026    |

Figura 18 — Tela de parceiros com a busca "praca" encontrando "Praça" — as primeiras doze linhas do resultado.

### 6.6 Documentação interativa

A especificação da API é gerada automaticamente e permite executar qualquer operação pelo navegador. É o contrato que a interface consome: a tela foi escrita contra ela, e não contra a implementação.

# Growth Intelligence Hub

0.2.0 OAS 3.1

/api/openapi.json

API do painel de inteligência de crescimento para redes de parceiros.

## autenticação

|        |                    |              |   |
|--------|--------------------|--------------|---|
| POST   | /api/sessao        | Autenticar   | ▼ |
| DELETE | /api/sessao        | Encerrar     | ▼ |
| GET    | /api/sessao/actual | Quem Sou     | ▼ |
| POST   | /api/sessao/senha  | Trocar Senha | ▼ |

## usuários

|       |                            |        |   |
|-------|----------------------------|--------|---|
| GET   | /api/usuarios              | Listar | ▼ |
| POST  | /api/usuarios              | Criar  | ▼ |
| GET   | /api/usuarios/{usuario_id} | Obter  | ▼ |
| PATCH | /api/usuarios/{usuario_id} | Editar | ▼ |

## parceiros

|        |                              |         |   |
|--------|------------------------------|---------|---|
| GET    | /api/parceiros               | Listar  | ▼ |
| POST   | /api/parceiros               | Criar   | ▼ |
| GET    | /api/parceiros/{parceiro_id} | Obter   | ▼ |
| PATCH  | /api/parceiros/{parceiro_id} | Editar  | ▼ |
| DELETE | /api/parceiros/{parceiro_id} | Excluir | ▼ |
| GET    | /api/categorias              | Listar  | ▼ |
| POST   | /api/categorias              | Criar   | ▼ |

## importação

|      |                                 |                   |   |
|------|---------------------------------|-------------------|---|
| POST | /api/importacoes/previa         | Previa            | ▼ |
| POST | /api/importacoes                | Importar          | ▼ |
| GET  | /api/importacoes                | Historico         | ▼ |
| POST | /api/importacoes/arquivo/previa | Previa De Arquivo | ▼ |
| POST | /api/importacoes/arquivo        | Importar Arquivo  | ▼ |

## painel

|     |                         |             |   |
|-----|-------------------------|-------------|---|
| GET | /api/painel/indicadores | Indicadores | ▼ |
| GET | /api/painel/ranking     | Ranking     | ▼ |
| GET | /api/painel/series      | Series      | ▼ |

## auditoria

|     |                      |                 |   |
|-----|----------------------|-----------------|---|
| GET | /api/auditoria       | Consultar       | ▼ |
| GET | /api/auditoria/acoes | Acoes Possiveis | ▼ |

## sistema

|     |             |        |   |
|-----|-------------|--------|---|
| GET | /api/health | Health | ▼ |
|-----|-------------|--------|---|

Figura 19 — Documentação interativa em /api/docs: as 26 operações, agrupadas por assunto.

GET

/api/parceiros

Listar

Lista com busca por nome e filtros (RF14, RF24).

**selectinload** na categoria não é detalhe: sem ele, montar a resposta faria uma consulta por linha para buscar o nome da categoria — o N+1 que funciona com 100 parceiros e morre com 10.000, que é a carga do RNF04.

Parameters

Try it out

| Name                                  | Description                                     |
|---------------------------------------|-------------------------------------------------|
| busca                                 | Trecho do nome. Ignora maiúsculas e acentuação. |
| string   (string   null)<br>(query)   | <input type="text" value="busca"/>              |
| categoria_id                          |                                                 |
| integer   (integer   null)<br>(query) | <input type="text" value="categoria_id"/>       |
| status                                | Available values : ATIVO, PROSPECCAO, INATIVO   |
| string   (string   null)<br>(query)   | <input type="text" value="--"/>                 |
| ativo                                 | Filtra por situação.                            |
| boolean   (boolean   null)<br>(query) | <input type="text" value="--"/>                 |
| sem_categoria                         | Só os pendentes de classificação.               |
| boolean   (boolean   null)<br>(query) | <input type="text" value="--"/>                 |
| gih_sessao                            |                                                 |
| string   (string   null)<br>(cookie)  | <input type="text" value="gih_sessao"/>         |

Responses

| Code | Description                                                                                                                                                                                                                                                                                                                                                                                                                             | Links    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 200  | Successful Response                                                                                                                                                                                                                                                                                                                                                                                                                     | No links |
|      | <div><div>Media type</div><div><input type="text" value="application/json"/></div><div>Controls Accept header.</div><div>Example Value   Schema</div><div><pre>{   "id": 0,   "nome": "string",   "categoria": {     "id": 0,     "nome": "string",     "ativa": true   },   "origem_categoria": "INFERIDA",   "status": "ATIVO",   "contato": "string",   "ativo": true,   "criado_em": "2026-09-19T01:10:44.985Z" }</pre></div></div> |          |
| 422  | Validation Error                                                                                                                                                                                                                                                                                                                                                                                                                        | No links |
|      | <div><div>Media type</div><div><input type="text" value="application/json"/></div><div>Example Value   Schema</div><div><pre>{   "detail": [     {       "loc": [         "string",         0       ],       "msg": "string",       "type": "string"     }   ] }</pre></div></div>                                                                                                                                                      |          |

Figura 20 — Contrato de entrada e saída da listagem de parceiros, com os filtros disponíveis.

## 7. Execução local

O ambiente completo — banco, API e interface — sobe com um comando, a partir de um clone limpo. Não há etapa manual de configuração: o processo de inicialização aplica as migrações e cria o administrador antes de servir a API.

### 7.1 Pré-requisitos

- Docker Desktop — é o único obrigatório
- Python 3.11 e Node.js 22, apenas para rodar a verificação e os testes fora do contêiner
- Placa NVIDIA e CUDA Toolkit são opcionais, e só para o núcleo computacional

### 7.2 Passo a passo

```
git clone https://github.com/PedroMiranda243/gih-fabrica-de-software.git
cd gih-fabrica-de-software
cp .env.example .env
docker compose up
```

O compose sobe o PostgreSQL, espera ele ficar saudável, aplica as migrações, cria o administrador inicial se não houver nenhum, serve a API e sobe a interface. A aplicação fica em <http://localhost:5173>.

A interface e a API dividem o mesmo endereço: o servidor da interface repassa o que é de /api. É o que deixa o cookie de sessão funcionar sem abrir a API a outras origens — a postura de segurança do servidor não precisou mudar para acomodar a tela.

### 7.3 Primeiro acesso

Crie o próprio usuário pelo terminal, com o ambiente no ar. A senha é digitada, duas vezes:

```
docker compose exec api python -m app.cli criar-usuario \
  --login seu.login --nome "Seu Nome"
```

O perfil padrão é Gestor, que é o que usa painel, importação e parceiros. Depois, entre em <http://localhost:5173>.

**Não existe senha padrão.** O repositório é público, e um usuário e senha fixos no código seriam porta aberta em qualquer instalação que esquecesse de trocá-los. O administrador inicial recebe uma senha sorteada, impressa uma única vez no log da primeira subida; perdida, ela é redefinida pelo comando `redefinir-senha`.

### 7.4 Verificando que subiu

| O quê                        | Onde                                                                            |
|------------------------------|---------------------------------------------------------------------------------|
| A aplicação                  | <a href="http://localhost:5173">http://localhost:5173</a>                       |
| Estado do sistema e do banco | <a href="http://localhost:8000/api/health">http://localhost:8000/api/health</a> |
| Documentação interativa      | <a href="http://localhost:8000/api/docs">http://localhost:8000/api/docs</a>     |
| Verificação de ponta a ponta | <code>cd api &amp;&amp; GIH_ADMIN_SENHA=... python e2e/verificacao.py</code>    |
| Suíte de testes              | <code>cd api &amp;&amp; pytest</code>                                           |

A verificação sai com código diferente de zero se qualquer checagem falhar — senão não é verificação, é impressão.

## 8. Repositório

Repositório: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

### 8.1 O que demonstra a evolução

- Histórias de usuário registradas como issues, agrupadas por sprint em milestones, com critérios de aceite verificáveis.
- Toda alteração entra por Pull Request revisado, com o dono de cada diretório convidado automaticamente.
- Integração contínua obrigatória: análise estática, suíte de testes contra um PostgreSQL real, e verificação de que nenhum dado ou segredo foi versionado.
- Documentação versionada junto com o código, atualizada no mesmo Pull Request que introduz a mudança.
- Este documento é gerado a partir da documentação do repositório, e não escrito à parte.

### 8.2 Divisão de responsabilidades

Cada diretório tem um dono. Isso não impede ninguém de contribuir — impede cinco pessoas de editarem o mesmo arquivo na mesma semana.

| Diretório       | Responsável          | Conteúdo                                         |
|-----------------|----------------------|--------------------------------------------------|
| <b>api/</b>     | Dev Backend / Núcleo | API, migrações e testes                          |
| <b>nucleo/</b>  | Dev Backend / Núcleo | Otimizador em C++ e CUDA                         |
| <b>web/</b>     | Dev Frontend         | Interface: login, painel, importação e parceiros |
| <b>docs/</b>    | Product Owner        | Documentação e este gerador                      |
| <b>scripts/</b> | Banco de Dados       | Gerador sintético e reinicialização              |
| <b>.github/</b> | Scrum Master         | Integração contínua                              |

## 9. Dificuldades encontradas

As dificuldades abaixo são reais, com a causa e a correção. Nenhuma delas é de prazo ou de coordenação: são problemas técnicos que custaram tempo e que ficaram registrados no repositório para não voltarem.

### 9.1 A GPU perde para a CPU paralela em escala pequena

A investigação técnica do núcleo mediu o ganho real e encontrou o contrário do esperado: abaixo de cerca de 4.000 planos candidatos, dezesseis threads de CPU batem a GPU inteira. Acima disso, a GPU com transferência a cada geração ganha apenas 1,1 vez do OpenMP — mas o kernel sozinho ganha 11 vezes. A transferência consome 88% do tempo.

**Correção:** manter a população na memória da GPU entre as gerações deixou de ser otimização e virou requisito de projeto do componente. Sem isso, o caminho CUDA não se justificaria.

### 9.2 O benchmark media o relógio, não o trabalho

Na validação do compilador, o ganho do OpenMP em 1.024 planos saiu 7,5 vezes numa execução e 13,9 vezes na seguinte — mesmo cenário, mesmo programa. Cinco repetições de uma operação que dura menos de um milissegundo medem a precisão do relógio, não o desempenho.

**Correção:** o teste passou a calibrar quantas repetições cabem num alvo de tempo, a aquecer antes de medir, e a reportar mediana de várias execuções. Um número que não se reproduz não serve de resultado.

### 9.3 O contêiner morria apontando para um arquivo que existe

O script de inicialização foi gravado com fim de linha do Windows, e o Docker respondeu que o arquivo não existe — apontando exatamente para o arquivo que estava lá. O comparador de versões não mostrava diferença nenhuma, porque ele normaliza a leitura enquanto o Docker copia o arquivo do disco.

**Correção:** o fim de linha passou a ser fixado no repositório por configuração, nos arquivos que o Linux precisa ler.

### 9.4 Os diagramas saíram vazios no PDF

Na entrega anterior, os diagramas foram embutidos em formato vetorial. O documento abria normal na tela, mas ao exportar para PDF o Word desenhava as caixas e as setas e descartava todo o texto dentro delas. Um diagrama de classes sem os nomes das classes não é um diagrama ruim — é uma figura vazia, e teria sido entregue assim.

**Correção:** os diagramas passaram a entrar rasterizados em resolução de impressão, e a conferência da entrega passou a incluir abrir o PDF e olhar as páginas — e não apenas conferir que o arquivo foi gerado.

### 9.5 A própria verificação reprovava por defeito dela

A verificação de ponta a ponta falhou duas vezes por causa de si mesma antes de passar. A varredura de autorização percorria o endereço que encerra a sessão, e tudo depois dele respondia como não autenticado — o sintoma parecia falha de permissão. E o período usado na importação era fixo, colidindo com a execução anterior.

**Correção:** a varredura passou a pular as operações destrutivas, e a marca de cada execução passou a ser sorteada em vez de derivada do relógio. Os dois motivos ficaram comentados no código.

### 9.6 A primeira versão da interface reprovou no contraste

O contraste foi medido, e não suposto: cada cor pintada e o pixel lido, nos dois modos. A primeira versão reprovou em três pontos. O pior foi o texto branco sobre o botão âmbar, a 2,15:1 no modo escuro — o mínimo exigido é 4,5:1. A mesma medição achou o número da variação pintado com a cor de um segmento, a 2,82:1, o que também violava a regra do sistema visual: cor de segmento só como ponto ou barra, nunca como texto.

**Correção:** texto escuro fixo sobre o âmbar (8,27:1 no escuro), texto neutro na variação com a cor só na seta, e a tinta mais clara restrita ao que não é texto. Relendo com a regra na cabeça, a cor de ação também saiu da sigla e do ícone, onde fazia papel de marca — o erro exato do projeto anterior.

### 9.7 A integração contínua rodava num Node sem suporte

A verificação da interface reprovou na primeira execução com um erro que não dizia nada sobre a causa: uma função ausente dentro de uma biblioteca. A integração contínua rodava o Node 20, e a cadeia de teste exige o 22 — na máquina de desenvolvimento, com o 24, tudo passava. O Node 20 saiu de suporte em abril de 2026.

**Correção:** a integração contínua passou para o Node 22, o mesmo da imagem em que a interface roda, em vez de rebaixar as bibliotecas para caber num ambiente sem correção de segurança. A mudança foi conferida num contêiner com o Node 22 antes de publicada.

## 9.8 A verificação sujava o banco em que se demonstra

Para não colidir com execuções anteriores, a verificação de ponta a ponta importava relatórios em períodos no futuro — e nunca os removía. Depois de algumas execuções, o painel, que abre no período mais recente, passou a abrir em 2055, com um parceiro só.

**Correção:** a verificação passou a apagar o que criou ao terminar, e a conferir isso como uma das próprias checagens: o painel precisa voltar a abrir no mesmo período, com os mesmos totais.

## 10. Ajustes no planejamento e na modelagem

A entrega pede que alterações feitas nas sprints anteriores sejam registradas e justificadas. Foram sete, e todas estão versionadas no repositório com a justificativa junto.

| O que mudou                    | De / para                                                     | Por quê                                                                                                                                                                                                                                                  |
|--------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cadência das sprints           | Quinzenal para semanal                                        | Pedido na avaliação da Sprint 01. O atraso passa a aparecer na primeira semana, não na terceira. Duas histórias grandes demais para uma semana foram quebradas; o total de pontos não mudou                                                              |
| Número de tabelas              | 14 para 16                                                    | A implementação da autenticação exigiu guardar a sessão e as tentativas de login. A sessão precisa de estado no servidor para que encerrar tenha efeito imediato                                                                                         |
| Compilação do núcleo           | Em tempo de execução para antecipada                          | O compilador C++ e o toolkit foram instalados, e a implementação em OpenMP exige compilação antecipada de qualquer forma. A decisão anterior ficou registrada como histórico                                                                             |
| Formato do relatório importado | Indefinido para tolerante                                     | A história pedia "o formato definido", e não havia formato definido em lugar nenhum. Foi decidido em equipe: separador livre, ordem de colunas livre, sinônimos aceitos                                                                                  |
| Exclusão de parceiro           | Apenas desativar para exclusão condicional                    | Esta entrega pede a operação de excluir. A exclusão acontece quando não há histórico, e é recusada com explicação quando há                                                                                                                              |
| Aplicação web                  | Sem história no backlog para uma história própria, antecipada | O backlog pressupunha telas em várias histórias, mas nenhuma mandava construir a aplicação — por isso o servidor avançou e a interface não. A lacuna virou história própria, e a interface foi antecipada da Sprint 6 assim que o protótipo foi aprovado |
| Acesso ao sistema              | Senha sorteada no log para comando de terminal                | Com a senha do administrador só no log da primeira subida, quem clonasse o projeto não conseguia entrar. Os comandos de criar usuário e redefinir senha tornaram o primeiro acesso reproduzível                                                          |

Um ponto que estava em aberto na versão anterior deste documento foi fechado: o critério de aceite da história do ambiente dizia que o comando único sobe também a interface, e essa metade não estava cumprida. Agora está — um comando sobe banco, API e interface.

E um ponto segue em aberto, declarado em vez de omitido: a tela de administração de usuários. O cadastro existe e funciona pela API e pelo terminal, mas a interface ainda não o oferece.

## 11. Próximos passos

A ordem abaixo segue o backlog priorizado da Sprint 01, com o ajuste de cadência já aplicado.

| Sprint | Tema                       | O que entra                                                                                |
|--------|----------------------------|--------------------------------------------------------------------------------------------|
| 6      | Ingestão completa e painel | Quase concluída: falta medir o painel com 5.000 parceiros, que é o requisito de desempenho |
| 7      | Segmentação e mobilidade   | Classificação determinística com precedência explícita, ranking e mobilidade do Top N      |
| 8      | Modelo preditivo           | Treino a partir do histórico, previsão de faturamento e probabilidade de queda             |
| 9 a 11 | Núcleo computacional       | Otimizador serial, paralelo em CPU e paralelo em GPU, com o comparativo de desempenho      |
| 12     | Central de comunicação     | Geração de mensagens por segmento, com aprovação humana obrigatória                        |
| 13     | Assistente e fechamento    | Perguntas em linguagem natural sobre dados já apurados, e a entrega final                  |

### 11.1 O que vem primeiro

- Medir o painel com 5.000 parceiros e registrar a mediana com a dispersão — é o que fecha a Sprint 6.
- A tela de administração de usuários, que é a lacuna declarada na seção 4.
- A segmentação determinística e a mobilidade do Top N, lida do ranking que já existe — a Sprint 7.
- A sugestão automática de categoria pelo nome do parceiro, que completa o cadastro.
- O recálculo da segmentação após cada importação, que hoje não acontece.

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*